GD30BM2016

3-16 串软件用户指南

GD30BM2016 器件上电后用户需要根据应用需求对其初始化，主要是对一些重要参数进行配置，其中包含系统参数（MOSFET 控制，采样速率，低功耗模式等），校准（如采样电阻值），引脚重新映射（NTC，GPIO 等），一次和二次保护参数以及均衡等。GD30BM2016 软件用户指南用于在器件系统开发过程中，指导用户进行软件配置，初始化流程，数据读取等。

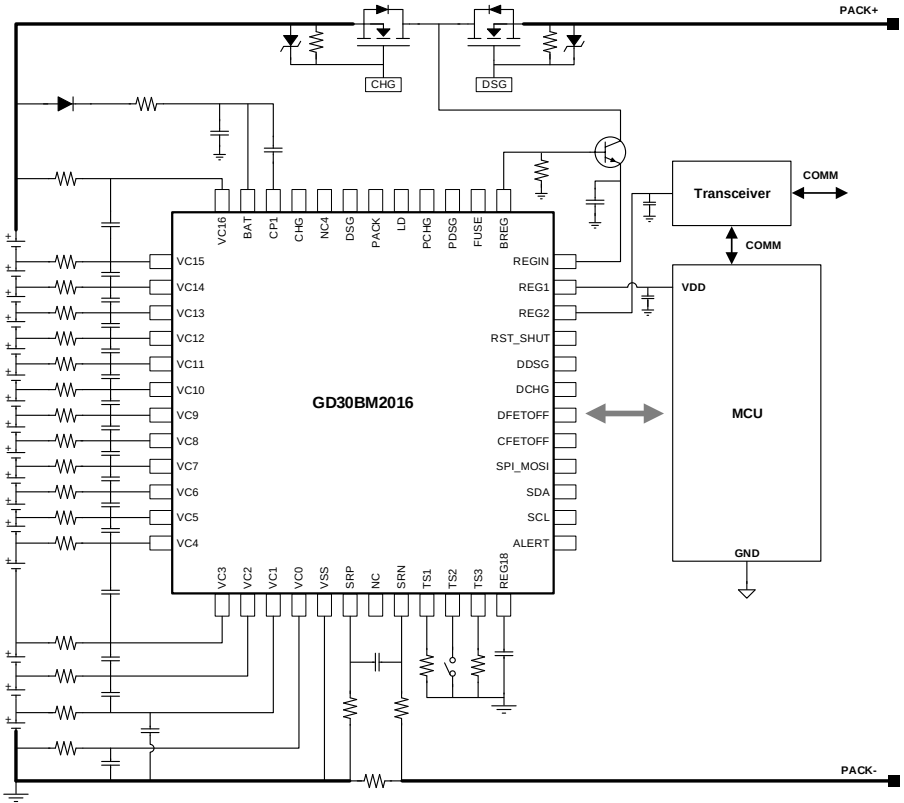
特性

- 精确的电流、电压、温度等监测能力
- 1.5V 至 5.5V 的单体电池电压范围
- 宽量程电流检测电压范围（±200mV）
- 多通道同时均衡，内置均衡 MOSFET 支持最大 80mA 均衡电流
- 生产线上编程的（OTP）存储器
- 提供 400kHz I2C 和 2MHz SPI 通信接口选项
- 恶劣环境下安全可靠运行：85V 的耐压、二级保护、电压温度电流等多种硬件保护机制等
- 多种低功耗运行模式

应用场景

- 便携式电源
- 低压家庭储能
- 锂电池两轮车
- 高压电动工具
- 其它工业电池组

GD30BM2016 简化原理图



Copyright @ 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

目录

1 引言 4
2 系统框图 4
3 直接命令 5
3.1 直接命令格式 5
3.2 直接命令表 7
3.3 直接命令案例 7
4 子命令 9
4.1 不带数据子命令格式 9
4.2 不带数据子命令表 9
4.3 不带数据子命令示例 9
4.4 带数据子命令格式 10
4.5 带数据子命令示例 12
5 读写 RAM 15
5.1 读写 RAM 格式 15
5.2 CRC 校验 16
5.3 Check Sum 计算 18
5.4 不带 CRC 读写寄存器示例 18
5.5 带 CRC 读写寄存器示例 21
6 OTP 读写 24
7 ADC 较准 25
7.1 电流校准 25
7.2 温度校准 25
8 器件使用流程 26
8.1 器件使用注意事项 26
8.2 初始化流程 26
8.3 初始化配置示例 27
8.4 采样数据和寄存器状态读取 30
8.5 采样数据转换计算 30
9 历史版本 32

Copyright @ 2025, GigaDevice Semiconductor Inc.
All Rights Reserverd.

插图清单

图 2-1 GD30BM2016 功能框图 4
图 3-1 Alarm Status 直接命令读取 7
图 3-2 Cell1 电压直接命令读取 8
图 3-3 内部温度直接命令读取 8
图 3-4 CC2 电流直接命令读取 8
图 4-1 CONFIG_UPDATE 子命令示例 10
图 4-2 EXIT_CFGUPDATE 子命令示例 10
图 4-3 DEVICE_NUMBER 子命令示例 13
图 4-4 Access Key 子命令示例 14
图 5-1 checksum 计算示例 18
图 5-2 写 RAM 寄存器示例 19
图 5-3 读 RAM 寄存器示例 21
图 5-4 带 CRC 写 RAM 寄存器示例 22
图 5-5 带 CRC 读 RAM 寄存器示例 23

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

1 引言

GD30BM2016 是一款高度集成的高精度电池监控器和保护器，该器件包含一个高精度监控系统和一个高度可配置的保护子系统，并支持自主式或主机控制型电池平衡。GD30BM2016 通过 I2C 接口或 SPI 接口主控器件通信，主控器件在 GD30BM2016 上电时，根据系统要求对该器件进行初始化流程配置，读取状态和数据结果。

本指南提供了 GD30BM2016 常用的通信数据包和序列示例。示例包括直接命令、子命令以及读取和写入 RAM 寄存器的位事务处理详细信息。另外还提供了初始化和读取数据的简单代码示例，包含常用的寄存器地址，用户可与 GD30BM2016 Register Maps 寄存器表等一起使用。

2 系统框图

GD30BM2016 是一款高度集成的电池监管、均衡和保护器，专为 3~16 串锂离子电池组和磷酸铁锂电池组设计。该器件具备高精度的电压、电流和温度测量功能，为主控处理器提供精准的数据支持。其内置高度可配置的保护子系统可提供多种保护功能，这些功能既能由设备自主触发和恢复，也能完全由主控处理器控制。

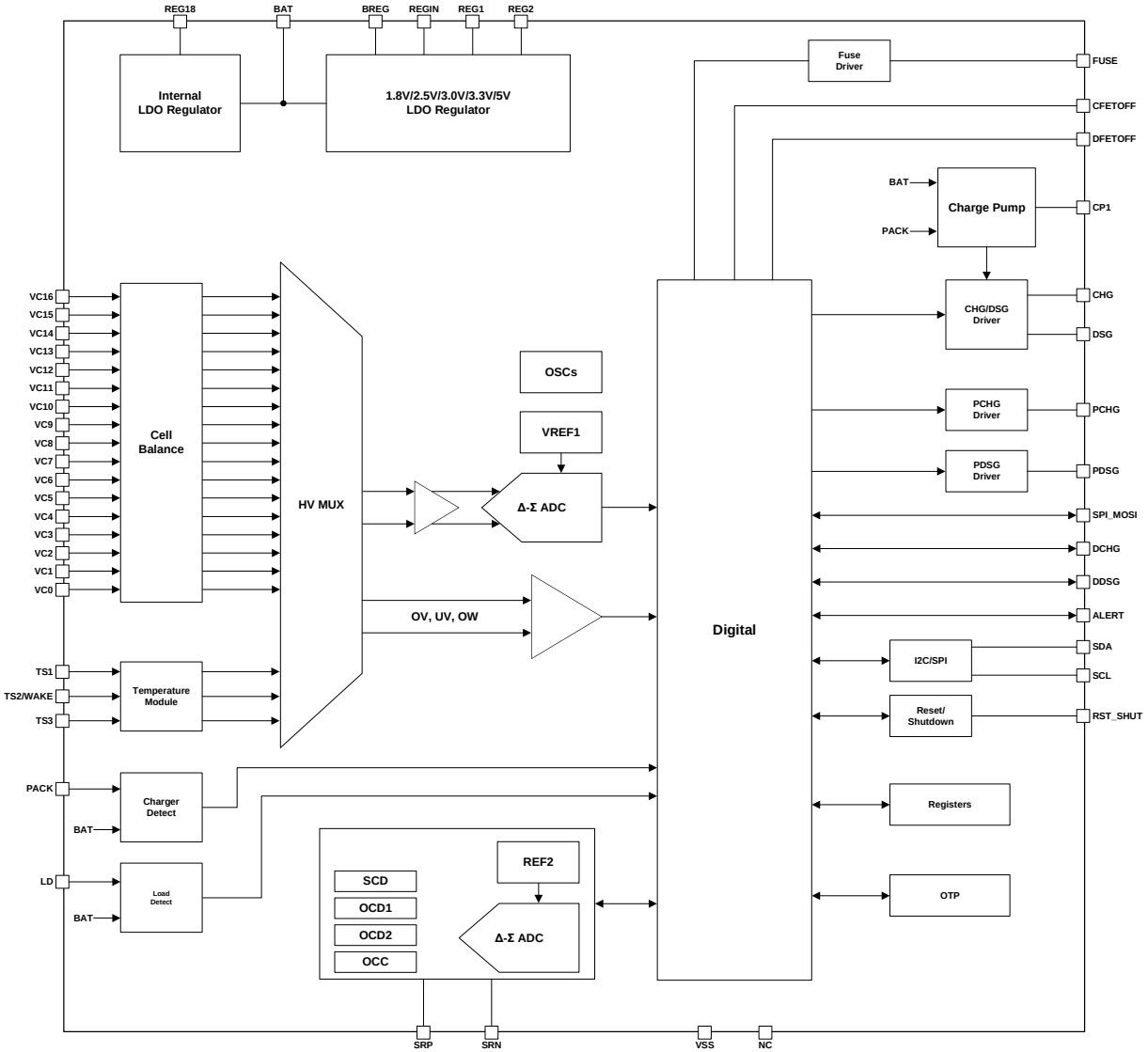


图2-1 GD30BM2016 功能框图

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

3 直接命令

GD30BM2016 支持两种串行通信接口，分别为 I2C、SPI 接口。I2C 接口支持最高 400kHz 时钟速率（可选 CRC 校验）；SPI 接口为从接口，支持高达 2MHz 的时钟速率（可选 CRC 校验）。GD30BM2016 不同器件版本支持不同通信模式，以及是否带有 CRC 校验。用户可以在生产线上对设备的集成 OTP 进行编程，以设置在设备上电运行时所需的通信速度和协议。GD30BM2016 在 I2C 模式下支持两个设备地址默认地址为 0x20，其设备地址由 SPI_MOSI 引脚决定，引脚 floating 或者拉低设备地址为 0x20，引脚拉高设备地址为 0x22。

GD30BM2016 器件支持直接命令和子命令，直接命令为单字节，子命令为两字节。直接命令使用 7 位地址直接访问器件，主机首先直接访问器件地址，待器件响应后写入直接命令。

3.1 直接命令格式

本章节为直接命令格式示例，蓝色为 MCU 发出的指令，白色为从机 GD30BM2016 应答，DirCMD 为直接命令，具体直接命令查阅*直接命令表*章节。

3.1.1 I2C 直接命令格式

下面为 I2C 的直接命令格式，其中：S 为 start 位， RS 为 restart 位，SLV_ADDR 代表 slave address 即 GD30BM2016 地址，W 为读写位中的写，A 为应答位表示主机或者从机 ack，N 为应答位表示主机或者从机 no ack，DirCMD 是直接命令 ID，BYTEx 是有效数据 Byte，P 为 stop 位。直接子命令中的 CRC 计算与寄存器的 CRC 计算原理一致，要送入计算的 CRC 数据也一致，具体计算请查阅 *CRC 校验*。

I2C 写指令：

S	SLV_ADDR	W	A	Dir CMD	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

I2C 读指令：

S	SLV_ADDR	W	A	Dir CMD	A	RS	SLV_ADDR	W	A	BYTE0	A	...	BYTEn	A	CRC8

3.1.2 SPI 直接命令

SPI 控制采用硬件片选信号，当片选信号 CS 拉低时允许 GD30BM2016 SPI 读写通信，拉高时禁止 GD30BM2016 SPI 读写通信。下面为 SPI 的直接命令格式，其中：cs_high 代表 SPI 的 CS 引脚拉高，cs_low 代表 CS 引脚拉低，DirCMD 是直接命令，BYTEx 是有效数据 Byte，dummy 代表空指令。SPI 通信时第一个字节为读写命令，该字节第 0 bit 为读写位，写命令时第 0bit 为 1，读命令时为 0，直接子命令左移一位后再或上读写位组成一个新的 8bit 数据。

SPI 写指令：

cs_high	cs_low						cs_high
	Dir CMD	W	BYTE0	...	BYTEn	CRC8	dummy

SPI 读指令：

cs_high	cs_low								cs_high
	Dir CMD	R	dummy	dummy	...	dummy	dummy		
	dummy		Dir CMD	R	BYTE0	...	BYTEn	CRC8	

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

3.2 直接命令表

直接命令通过主控制器发送的 7-bit 命令地址来访问，该命令可以触发一个动作、向器件写入数据，或者读取器件数据，下面为直接命令表，具体直接命令字段定义介绍查阅 *AN-GD30BM2016 Commands and Subcommands* 文档获取对应的直接子命令，按照上述格式向 GD30BM2016 发送直接命令。

3.3 直接命令案例

直接子命令中的 CRC 计算与寄存器的 CRC 计算原理一致，具体 CRC 计算请查阅 *CRC 校验*章节。

3.3.1 警报状态获取

Alarm Status 指令为 0x62，图 3-1 展示了使用 0x62 的警报状态读取命令。GD30BM2016 设备写地址为 0x20，设备读地址为 0x21，I2C 函数则是根据直接命令按照 I2C read 格式执行，读回来的数据为小端格式，低字节前高字节后，因此返回值为 0xD000。SPI 通信时第一个字节 0xc4(0x62<<1 | 00)，SPI MOSI 在读指令后需要再发三个 dummy 数据，允许 SPI_MISO 返回 3 个字节数据（读指令+读数据），数据以小端格式返回，返回 0x0000。

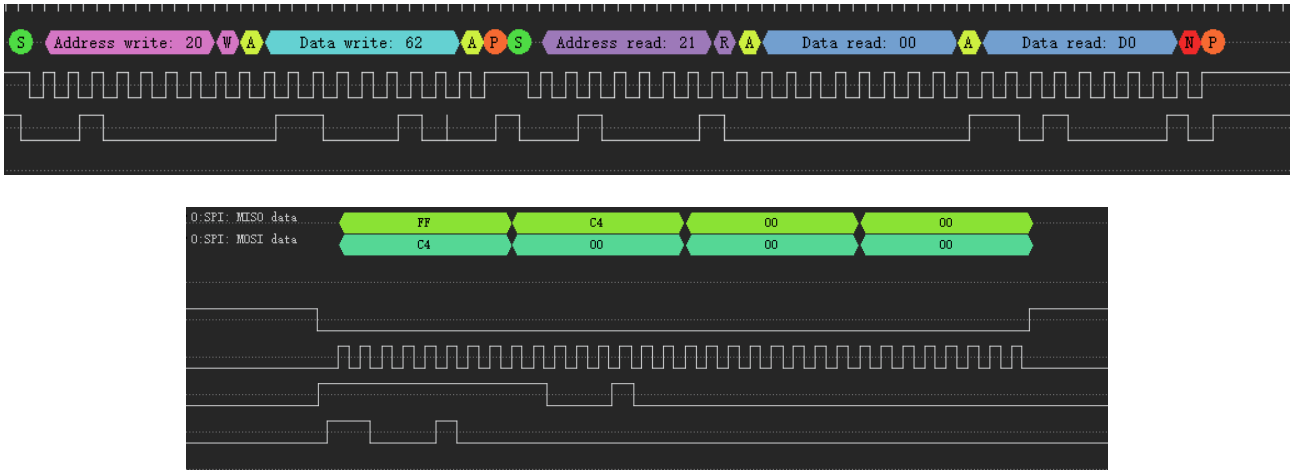
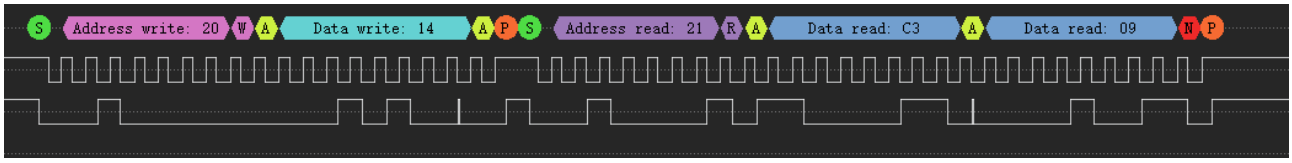


图3-1 Alarm Status 直接命令读取

3.3.2 电池 cell 电压的获取

cell 1 voltage 指令为 0x14，图 3-2 分别展示了 I2C 和 SPI 如何读取 cell 1 的电压。按照直接命令 I2C 格式写入器件地址后，写入直接命令 0x14，RS 后发送器件地址+读获取 cell1 电压测量数据，数据以小端格式返回，读取 16 位 Cell 1 电压为 0x09C3，单位为 mV 转换为 10 进制即 2499mV。SPI 通信时第一个字节 0x28(0x14<<1 | 00)，因为读 2 个字节，所以再发送读指令后需要再发三个 dummy 数据，数据以小端格式返回，读取 16 位 Cell 1 电压为 0x09C1，单位为 mV 转换为 10 进制即 2497mV。



Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

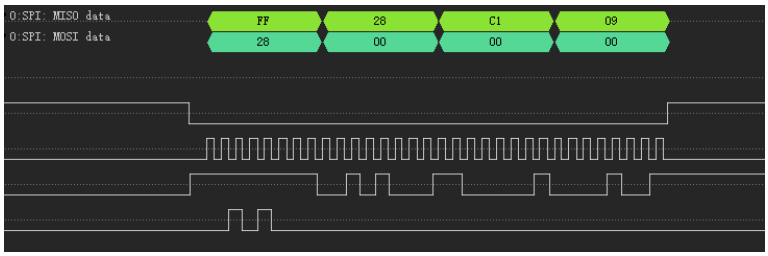


图3-2 Cell1 电压直接命令读取

3.3.3 温度的获取

获取内部温度直接指令（0x68），图3-3 表示读取内部温度传感器的通信命格式。16 位温度传感器的读数单位为 0.1°C。在以下示例中，0x0137 的读数表示十进制值 311，转换后约为 31.1°C。SPI 数据也以小端格式返回，读取 16 位温度为 0x0112，转为为 27.4°C。

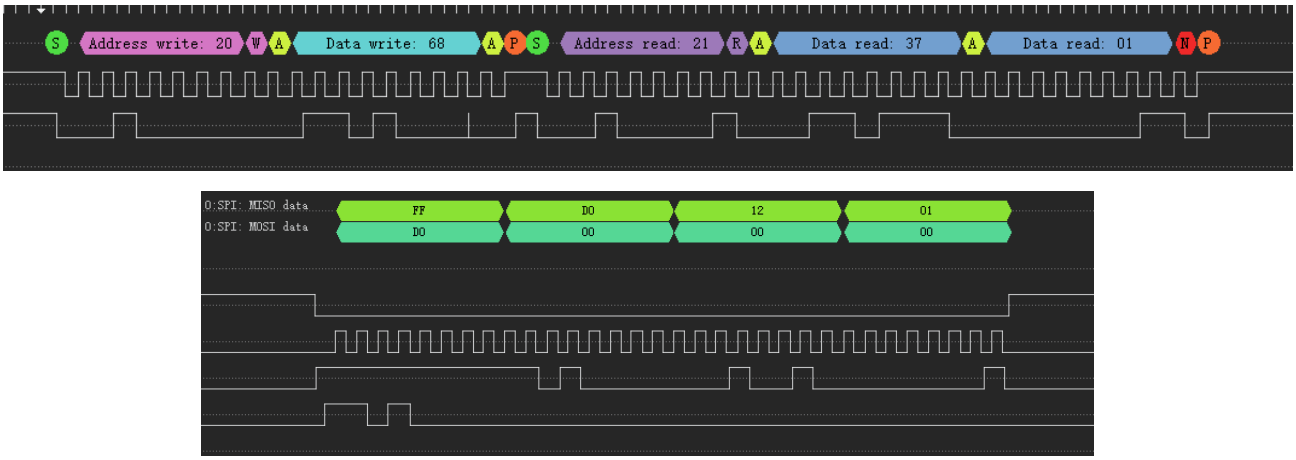


图3-3 内部温度直接命令读取

3.3.4 电流的获取

获 CC2 电流-直接指令为 0x3A 示出了如何从 CC2 读取 16 位电流测量值，I2C 示例中的电流读数 0x0083，单位为 userA，UserA 在 CC gain 中定义，用户根据采样电阻和最大电流设置合适 userA 的值，该示例中 userA 为 1mA,本次转换后电流为 130mA。SPI 数据也以小端格式返回，读取 16 位电流为 0xFFFD，转换为电流为-3mA。

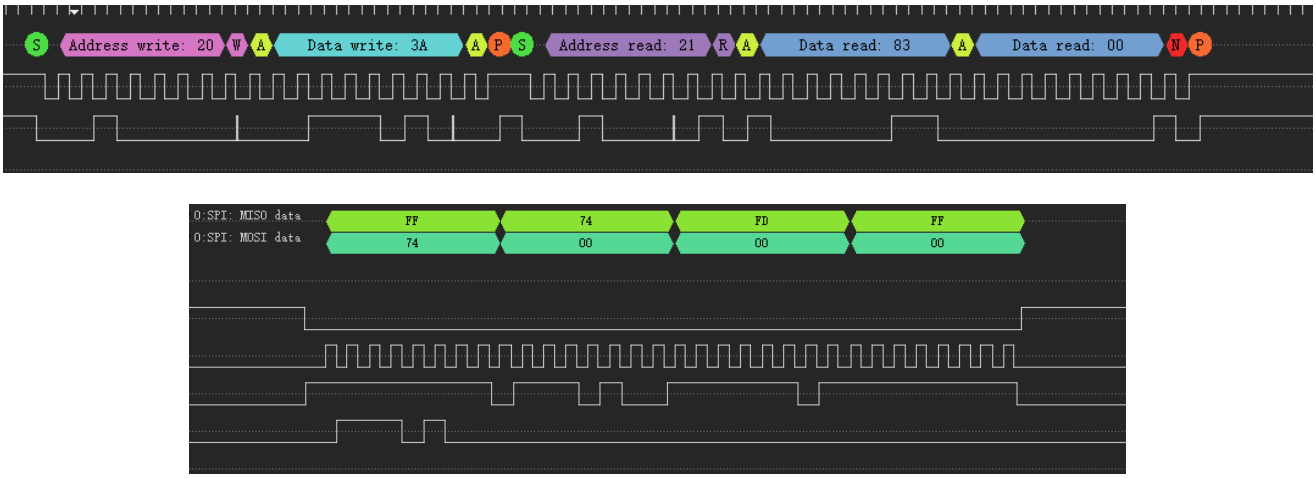


图3-4 CC2 电流直接命令读取

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

4 子命令

子命令为使用 7 位命令地址空间进行间接访问的附加命令，并提供了块数据传输功能。用户若使用子命令时，需先将 16 位子命令地址分别写入 0x3E(低字节)和 0x3F（高字节）。若此时为读命令，器件会填充对应的数据到 32 字节的缓冲区中，缓冲区地址为 0x40-0x5F。器件获取数据需要等待足够的时间，因此从子命令读取数据时，在写入 0x61 之后，等至少 100us，再从传输缓冲区读取结果。

子命令分为带数据的子命令和不带数据的子命令，编程时根据每种命令格式从上到下依次对 GD30BM2016 寄存器进行读写，如果不使用 CRC8 校验直接忽略 CRC 位即可。子命令中的 CRC 计算与寄存器的 CRC 计算原理一致，要送入计算的 CRC 数据也一致，具体 CRC 计算请查阅 *CRC 校验*。Checksum 的计算也与写寄存器的 checksum 一致，具体计算查阅 *Check Sum 计算*章节。

4.1 不带数据子命令格式

下面为不带数据子命令格式示例，蓝色为 MCU 发出的指令，白色为从机 GD30BM2016 应答，0x3E 为子命令入口地址，subcmd_l 为子命令低字节，subcmd_h 为子命令高字节，CRC8 为校验和，用户可以发送不带数据的子命令控制器件。

I2C 写

S	SLV_ADDR	W	A	0x3E	A	subcmd_l	A	subcmd_h	A	CRC8	N	P

SPI 写

cs_high	cs_low						cs_high
	0x3E	W	subcmd_l	subcmd_h	CRC8	dummy	
	dummy		0x3E	W	subcmd_l	subcmd_h	CRC8

4.2 不带数据子命令表

子命令操作向 0x3E/0X3F 入口地址写对应的子命令，例如向 0x3E（低字节）和 0x3F（高字节）写入 0x0304 命令，GD30BM2016 进入 DEEPSLEEP，子命令 ID 与 BQ76952 不同，用户严格按照下表发送子命令 ID。不带数据子命令字段定义查阅 *GD30BM2016 Commands and Subcommands* 手册。

4.3 不带数据子命令示例

不带数据子命令中的 CRC 计算与寄存器的 CRC 计算原理一致，要送入计算的 CRC 数据也一致，具体 CRC 计算请查阅 *CRC 校验*章节。

CONFIG_UPDATE 为不带数据子命令，用户写入 RAM 寄存器之前，需进入 CONFIG_UPDATE 模式，图 4-1 为进入 CONFIG_UPDATE 模式的子命令(I2C 和 SPI)，0x3E 为子命令入口地址，0x0338 为 CONFIG_UPDATE 子命令。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

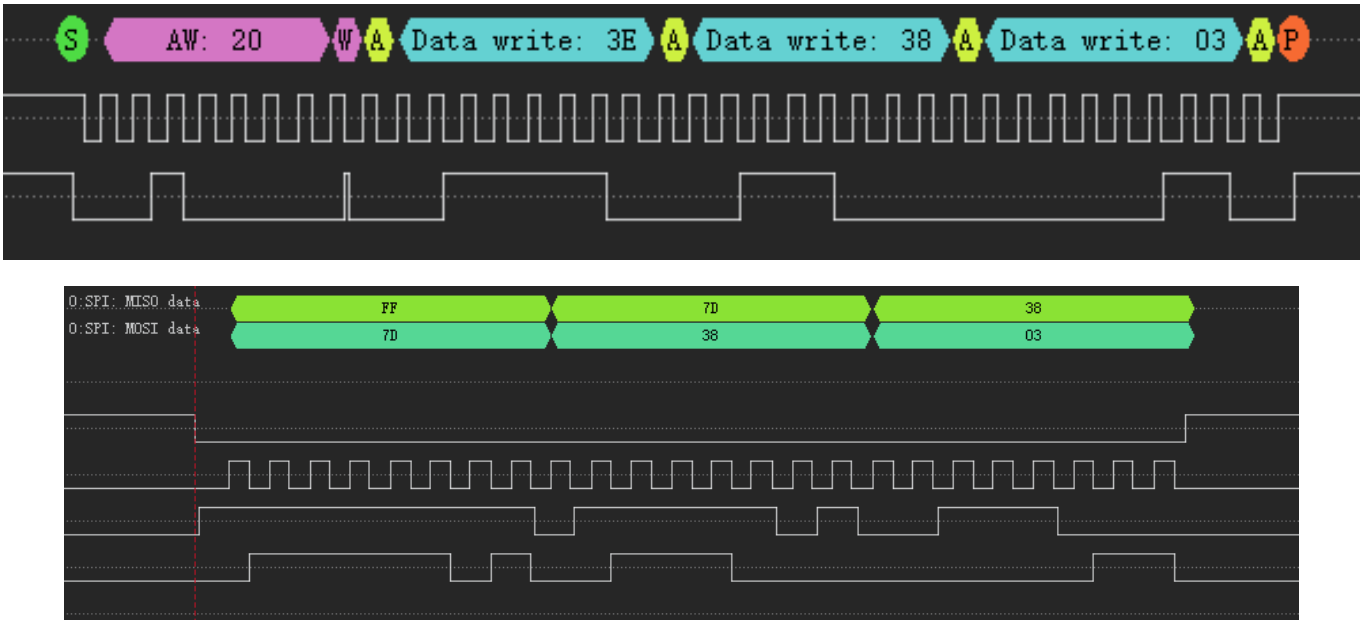


图4-1 CONFIG_UPDATE 子命令示例

写入 RAM 寄存器后，需退出 CONFIG_UPDATE 模式，此时寄存器新设置将生效。图 4-2 为退出 CONFIG_UPDATE 模式的子命令(I2C 和 SPI)，0x3E 为子命令入口地址，0x033C 为 EXIT_CFGUPDATE 子命令 ID。

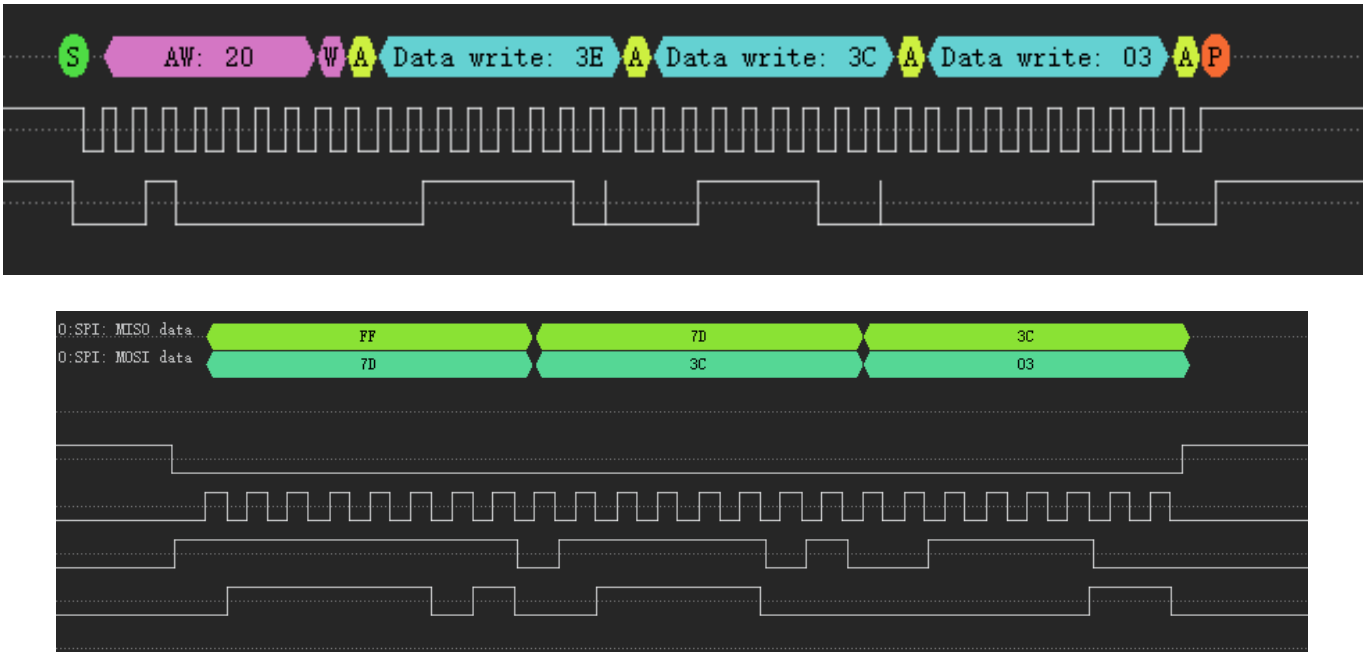


图4-2 EXIT_CFGUPDATE 子命令示例

4.4 带数据子命令格式

带数据的子命令格式如下所示，用户需按照该格式发送带数据的子命令控制器件。首先将 16 位子命令地址写入 0x3E（低位字节）和 0x3F（高位字节），并将现有数据自动填充到 32 字节的传输缓冲区 0x40–0x5F，该数据的校验和数据长度写入地址 0x60。checksum 校验和 CRC 计算在 *CRC 校验*和 *Check Sum 计算*章节中有更多描述。

I2C 写

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

S	SLV_ADDR	W	A	0x3E	A	subcmd_l	A	subcmd_h	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x60	A	checksum	A	len(W)	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x40	A	BYTE0	A	...	A	BYTEn	A	CRC8	N	P

I2C 读

S	SLV_ADDR	W	A	0x3E	A	subcmd_l		A	subcmd_h		A	CRC8	N	P		
S	SLV_ADDR	W	A	0x61	A	len(R)		A	CRC8		A	P				
S	SLV_ADDR	W	A	0x40	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P
S	SLV_ADDR	W	A	0x60	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

SPI 写格式

cs_high	cs_low						cs_high
	0x3E	W	subcmd_l	subcmd_h	CRC8	dummy	
	dummy		0x3E	W	subcmd_l	subcmd_h	CRC8

cs_high	cs_low						cs_high
	0x60	W	checksum	len(W)	CRC8	dummy	
	dummy		0x60	W	checksum	len(W)	CRC8

cs_high	cs_low						cs_high
	0x40	W	BYTE0	...	BYTEn	CRC8	
	dummy		0x40	W	BYTE0	...	BYTEn

SPI 读格式

cs_high	cs_low						cs_high
	0x3E	W	subcmd_l	subcmd_h	CRC8	dummy	
	dummy		0x3E	W	subcmd_l	subcmd_h	CRC8

cs_high	cs_low						cs_high
	0x61	W	checksum	len(W)	CRC8	dummy	
	dummy		0x61	W	checksum	len(W)	CRC8

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

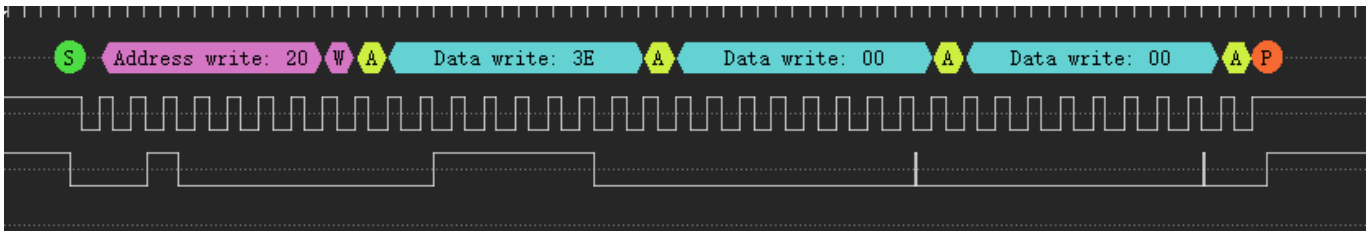
cs_high	cs_low							cs_high
	0x40	R	dummy	dummy	…	dummy	dummy	
	dummy		0x40	R	BYTE0	…	BYTEn	CRC8

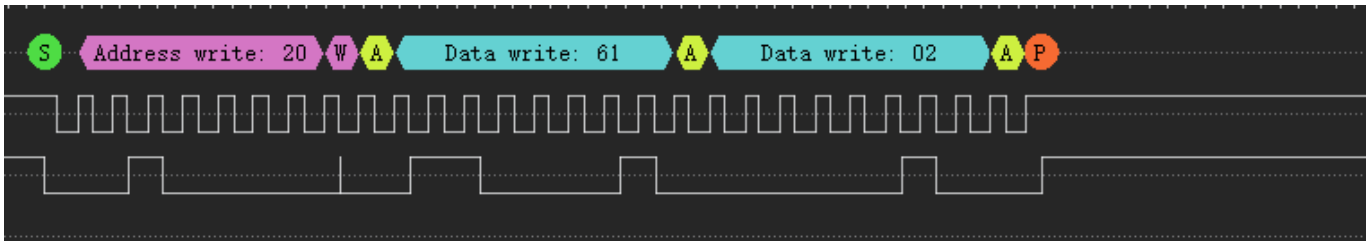
cs_high	cs_low							cs_high
	0x60	R	dummy	dummy	…	dummy	dummy	
	dummy		0x60	R	BYTE0	…	BYTEn	CRC8

4.5 带数据子命令示例

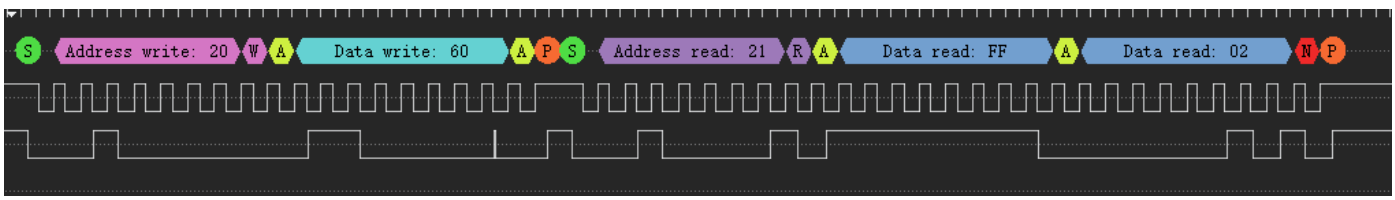
带数据子命令中的 CRC 计算与寄存器的 CRC 计算原理一致，要送入计算的 CRC 数据也一致，具体 CRC 计算请查阅 *CRC 校验*和 *Check Sum 计算*章节。

器件型号 DEVICE_NUMBER 子命令（0x0000）为带数据子命令读，I2C 或 SPI 通信通过将子命令编号 0x0000（小端格式）写入命令地址 0x3E 中来读取器件型号。将读取长度写入 0x61 寄存器中，然后从地址 0x40 的数据缓冲区中读取数据，从地址 0x60 的数据缓冲区读取校验和长度。在该示例中，返回的器件型号为 0x0000（代表 GD30BM2016）。









Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

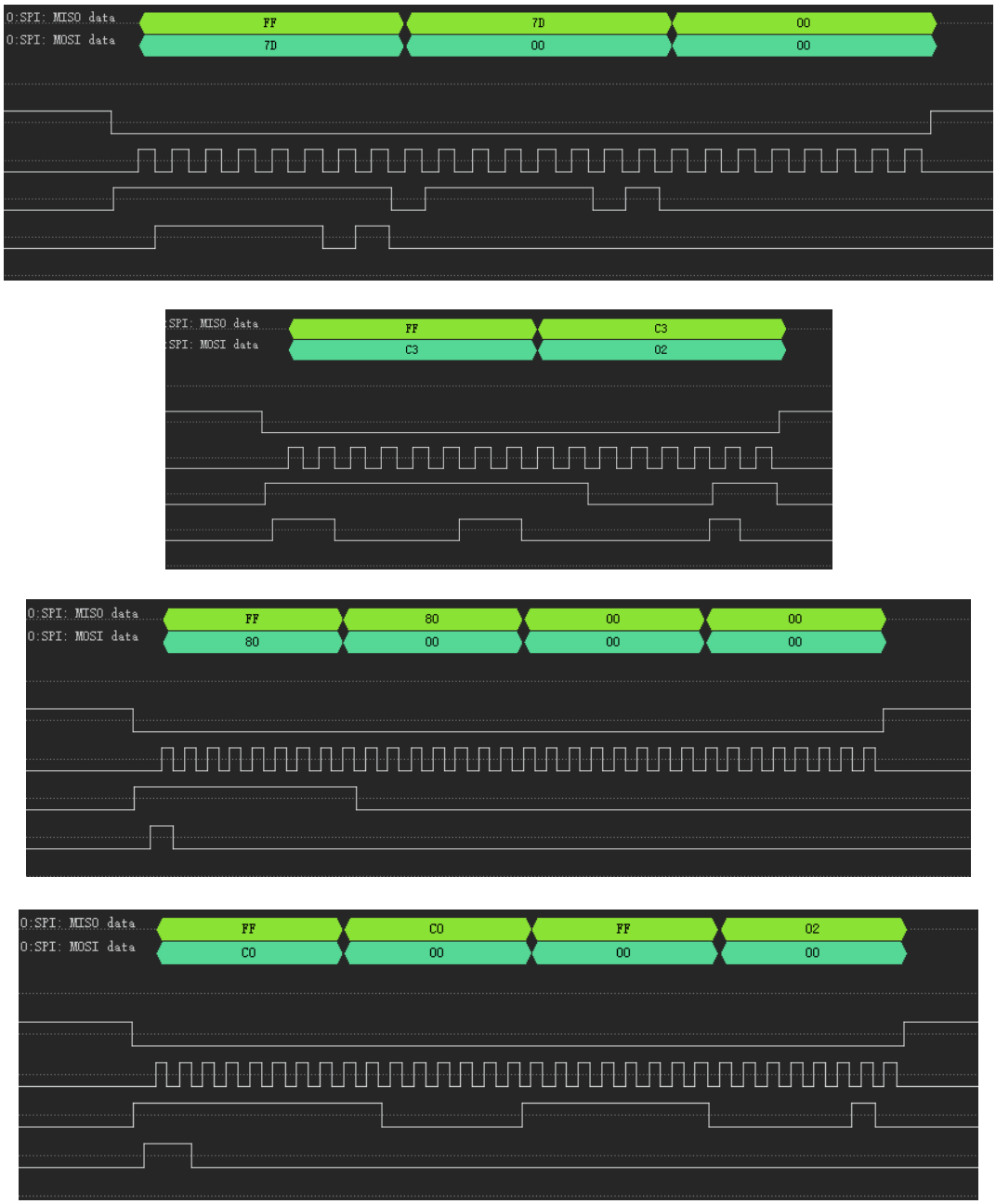
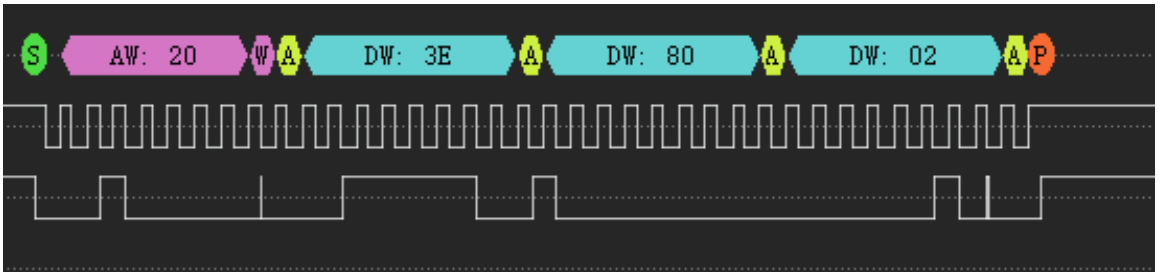
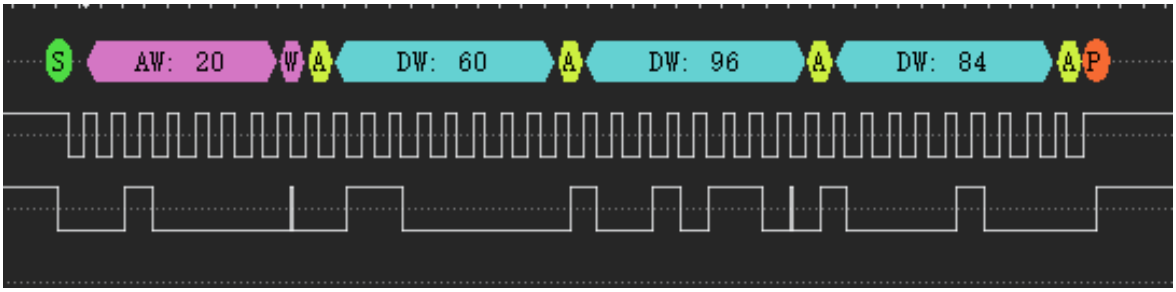


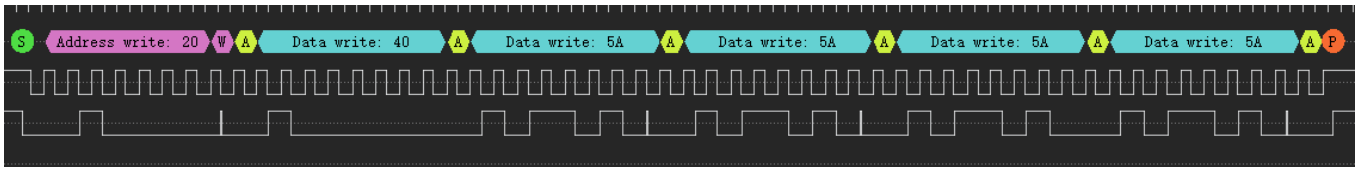
图4-3 DEVICE_NUMBER 子命令示例

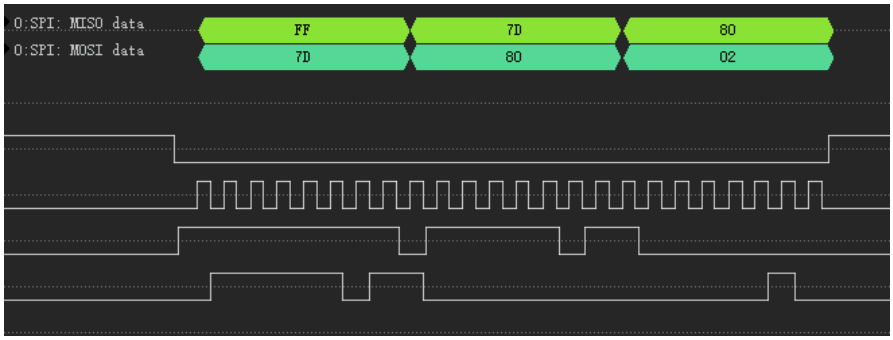
图 4-4 为 I2C 和 SPI Access key 写密钥子命令，进入 UNSEAL 模式先向 0x3E 入口地址写入子命令 0x0280，0x60 寄存器中写入 checksum 校验值和数据长度，写入长度字节计算方式为 0x80|写入长度，最后在缓冲寄存器 0x40-0x5F 写入密钥数据 0x5A5A5A5A。

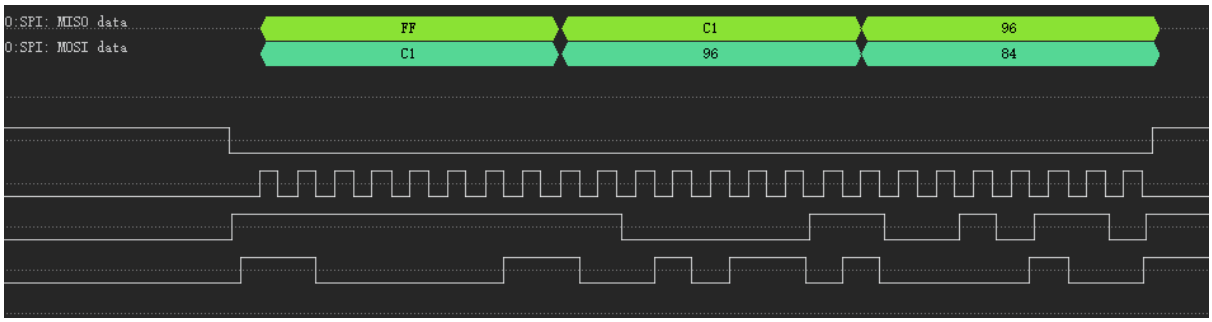


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.









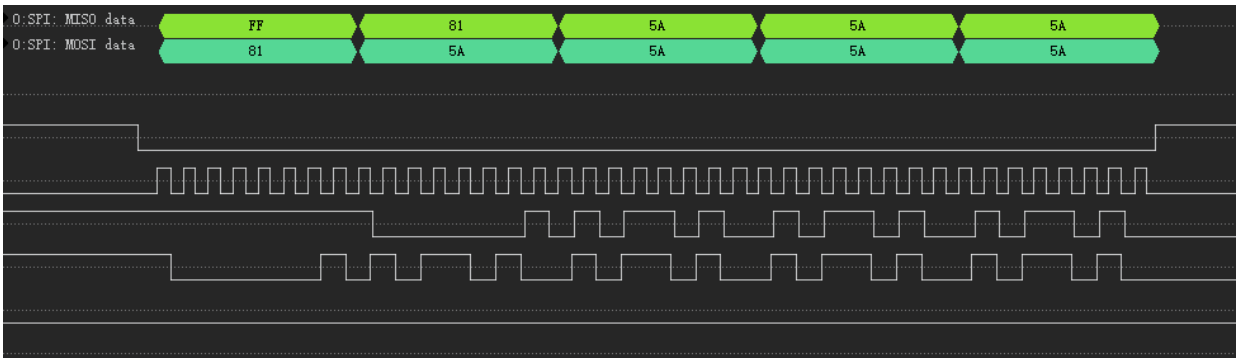


图4-4 Access Key 子命令示例

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

5 读写 RAM

将寄存器地址写入 0x3E，然后从 0x40 的数据缓冲区开始读取，以完成 RAM 寄存器的读取，其中校验和长度计算具体查阅 *CRC 校验*和 *Check Sum 计算*章节。写入 RAM 寄存器时，先进入 CONFIG_UPDATE 模式，写完成后执行退出 CONFIG_UPDATE 模式的命令，如此可以确保在修改设置时仍然稳定运行。寄存器字段定义请查阅 GD30BM2016 Register Maps Rev.A 文档。

5.1 读写 RAM 格式

I2C 写（注意 checksum 和 len 都只针对需要读写的数据）

S	SLV_ADDR	W	A	0x3E	A	addr_l	A	addr_h	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x60	A	checksum	A	len(W)	A	CRC8	N	P	
S	SLV_ADDR	W	A	0x40	A	BYTE0	A	...	A	BYTEn	A	CRC8	N	P

I2C 读

S	SLV_ADDR	W	A	0x3E	A	addr_l	A	addr_h	A	CRC8	N	P
S	SLV_ADDR	W	A	0x61	A	len(R)	A	CRC8	A	P		

S	SLV_ADDR	W	A	0x40	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P
S	SLV_ADDR	W	A	0x60	A	RS	SLV_ADDR	R	A	BYTE0	A	...	BYTEn	A	CRC8	N	P

SPI 写

cs_high	cs_low					cs_high
	0x3E	W	addr_l	addr_h	CRC8	dummy
	dummy	0x3E	W	addr_l	addr_h	CRC8

cs_high	cs_low					cs_high
	0x60	W	checksum	len(W)	CRC8	dummy
	dummy	0x60	W	checksum	len(W)	CRC8

cs_high	cs_low					cs_high
	0x40	W	BYTE0	...	BYTEn	CRC8
	dummy	0x40	W	BYTE0	...	BYTEn

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

SPI 读

cs_high	cs_low						cs_high
	0x3E	W	addr_l	addr_h	CRC8	dummy	
	dummy		0x3E	W	addr_l	addr_h	CRC8

cs_high	cs_low						cs_high
	0x61	W	checksum	len(W)	CRC8	dummy	
	dummy		0x61	W	checksum	len(W)	CRC8

cs_high	cs_low							cs_high
	0x40	R	dummy	dummy	…	dummy	dummy	
	dummy		0x40	R	BYTE0	...	BYTEn	CRC8

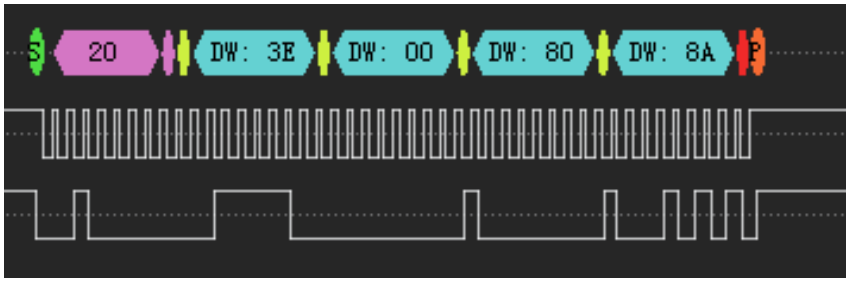
cs_high	cs_low							cs_high
	0x60	R	dummy	dummy	…	dummy	dummy	
	dummy		0x60	R	BYTE0	...	BYTEn	CRC8

5.2 CRC 校验

CRC 多项式为 x8 + x2 + x + 1，初始值为 0。当器件到错误的 CRC 时，GD30BM2016 禁止更新对应寄存器的值，下面介绍了 I2C 和 SPI 的 CRC 计算：

I2C写寄存器时包含单字节CRC计算和块字节CRC计算，具体为：

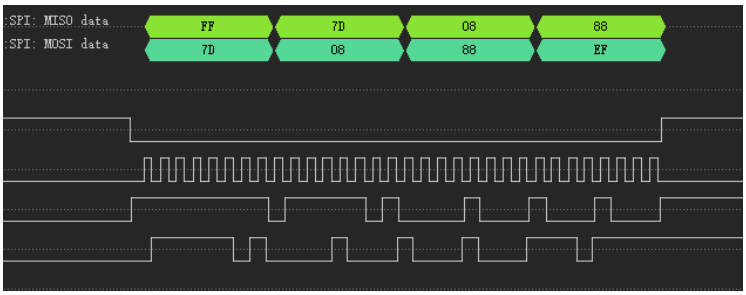
- 单字节写入事务中，根据器件地址、寄存器地址和所有写入数据来计算CRC，也就是说从start的第一个字节到最后写数据的最后一个有效字节均代入CRC计算中。
- 在块写入事务中，根据器件地址、寄存器地址和所有写入数据来计算CRC，从start的第一个字节到最后写数据的所有字节均代入CRC计算中，CRC为最后一个字节，写寄存器地址/command、写长度和checksum、写入缓冲区数据三条指令计算方式均一致。
- 下图为I2C带CRC写示例，0x20、0x3E、0x00、0x80为送入CRC计算的数据，0x8A为CRC结果，按照多项式为x8 + x2 + x + 1计算获得。



Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

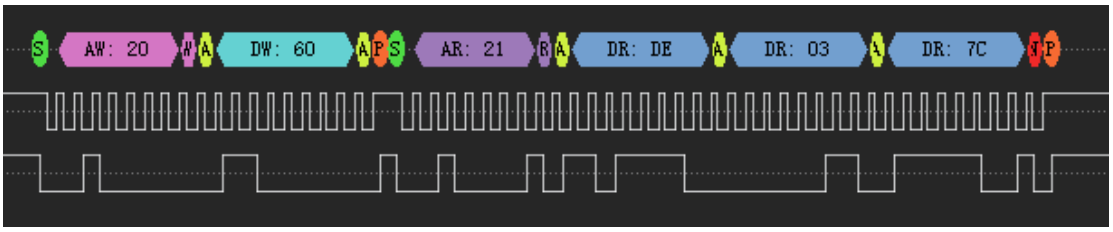
SPI写寄存器时包含单字节CRC计算和块字节CRC计算，具体为：

- 在单字节写入事务中，根据写命令和所有写数据来计算CRC。
- 在块写入事务中，根据写命令和所有写数据来计算CRC，第一个字节到最后写数据的所有字节均代入CRC计算中，CRC为最后一个字节，写寄存器地址/command、写长度和checksum、写入缓冲区数据三条指令计算方式均一致。
- 下图为SPI带CRC写示例，0x7D、0x08、0x88为送入CRC计算的数据，0xEF为CRC结果，按照多项式为 x8 + x2 + x + 1计算获得。



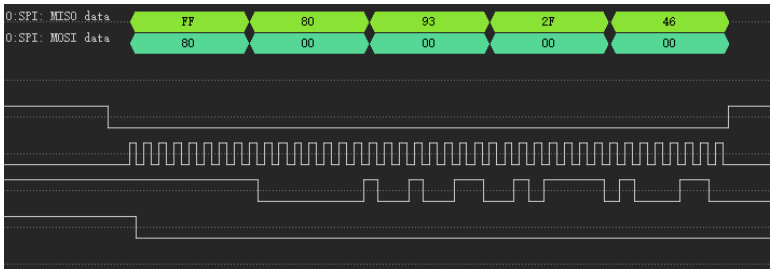
I2C读寄存器时通过以下方式来计算读取命令的 CRC：

- 在单字节读取事务中，CRC 是从第一个启动开始计算的，因此将包含器件地址、寄存器地址、然后是读器件地址，再是返回数据，即从start开始的第一个字节直到返回数据的最后一个字节均送入到CRC计算中。
- 在块读取事务中，CRC是从第一个启动开始计算的，将包含器件地址、寄存器地址、然后是读器件地址，再是返回数据，从start开始的字节直到返回数据的最后一个字节均送入到CRC计算中，在stop之后crc复位并在下一次通信重新计算。
- 下图为I2C带CRC读示例，0x20、0x60、0x21、0xDE、0x03送入CRC计算的数据，0x7C为CRC结果，按照多项式为 x8 + x2 + x + 1计算获得。



SPI读寄存器时可以通过以下方式来计算读取命令的 CRC：

- 与I2C不同的是，在单字节读取事务中，从第一个字节开始直到返回数据的倒数第二个字节均送入到CRC计算中，返回的AFE数据最后一个字节不送入CRC计算。
- 在块读取事务中，从第一个字节开始直到返回数据的倒数第二个字节均送入到CRC计算中，返回的AFE数据最后一个字节不送入CRC计算。
- 下图为SPI带CRC读示例，0x80、0x93送入CRC计算的数据，0x2F不计算CRC，0x46为CRC结果，按照多项式为 x8 + x2 + x + 1计算获得。



Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

5.3 Check Sum 计算

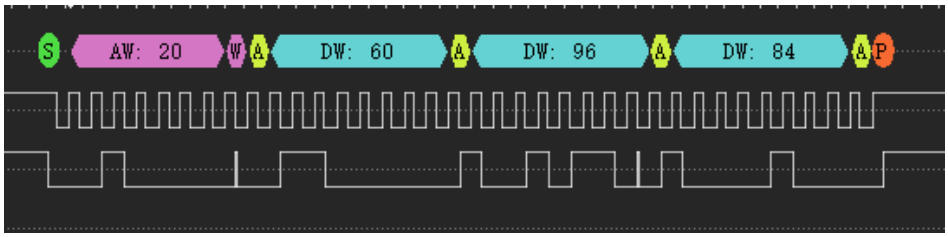
写寄存器时，需要再 0x60 后填入 checksum 数据，该数据是用来校正块地址写入数据，计算方法为累加块地址寄存器中写入的所有数据，计算得到数据和后，将高 8 位右移后再与第八位相加，循环该步骤直到结果小于等于 0xFF，再用 0xFF 减去该结果最终所得为 checksum 校验值，图 5-1 为 checksum 计算示例。

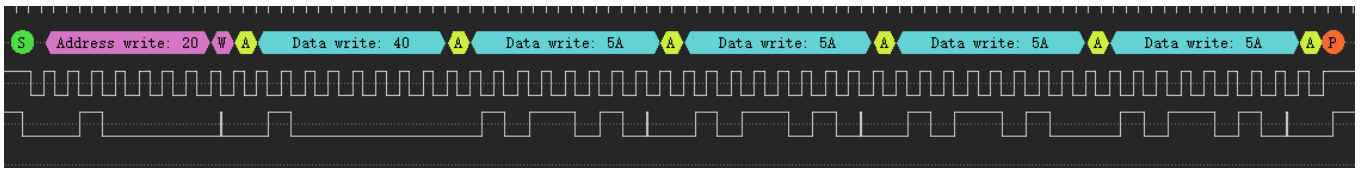
```
unsigned char Checksum(unsigned char *ptr, unsigned char len)
{
  unsigned char i=0;
  uint32_t checksum = 0;

  while(i<len){
    checksum =checksum + ptr[i];
    i++;
  }
  while (checksum>0xff)
  {
    checksum = (checksum >> 8) + (checksum & 0x000000ff);
  }
  checksum = 0xff - checksum;
  return(checksum);
}
```

图5-1 checksum 计算示例

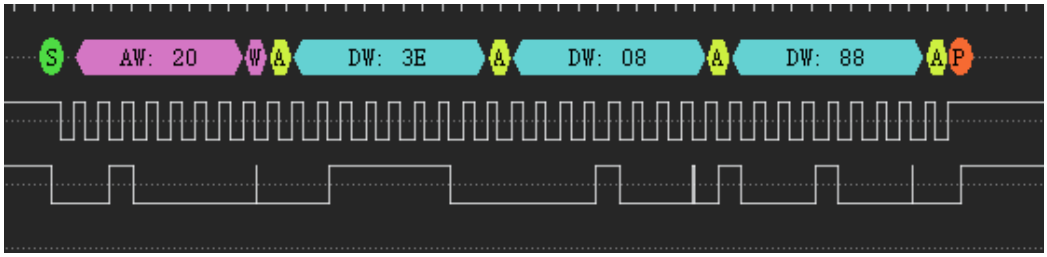
例如要连续写入 4 个 0x5A、0x5A、0x5A、0x5A 到数据缓冲区 0x40-0x5F，将要写入到 0x40-0x5F 缓冲区的 0x5A、0x5A、0x5A、0x5A 数据代入到 checksum 计算，计算结果为 0x96。由于要写 4 个长度，其要写入的长度数据为 0x84，计算方式为 0x80 | 写入长度。





5.4 不带 CRC 读写寄存器示例

图 5-2 为 Cell model2C 和 SPI 写格式，配置电池串数 16 串即向 0x8808 寄存器写入 0xFEFF 数据，通过将寄存器地址 0x8808（小端格式）写入 0x3E 入口地址中，0x60 寄存器中写入 chencksum 校验值和数据长度，最后在缓冲寄存器 0x40-0x5F 写入对应的数据。



Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

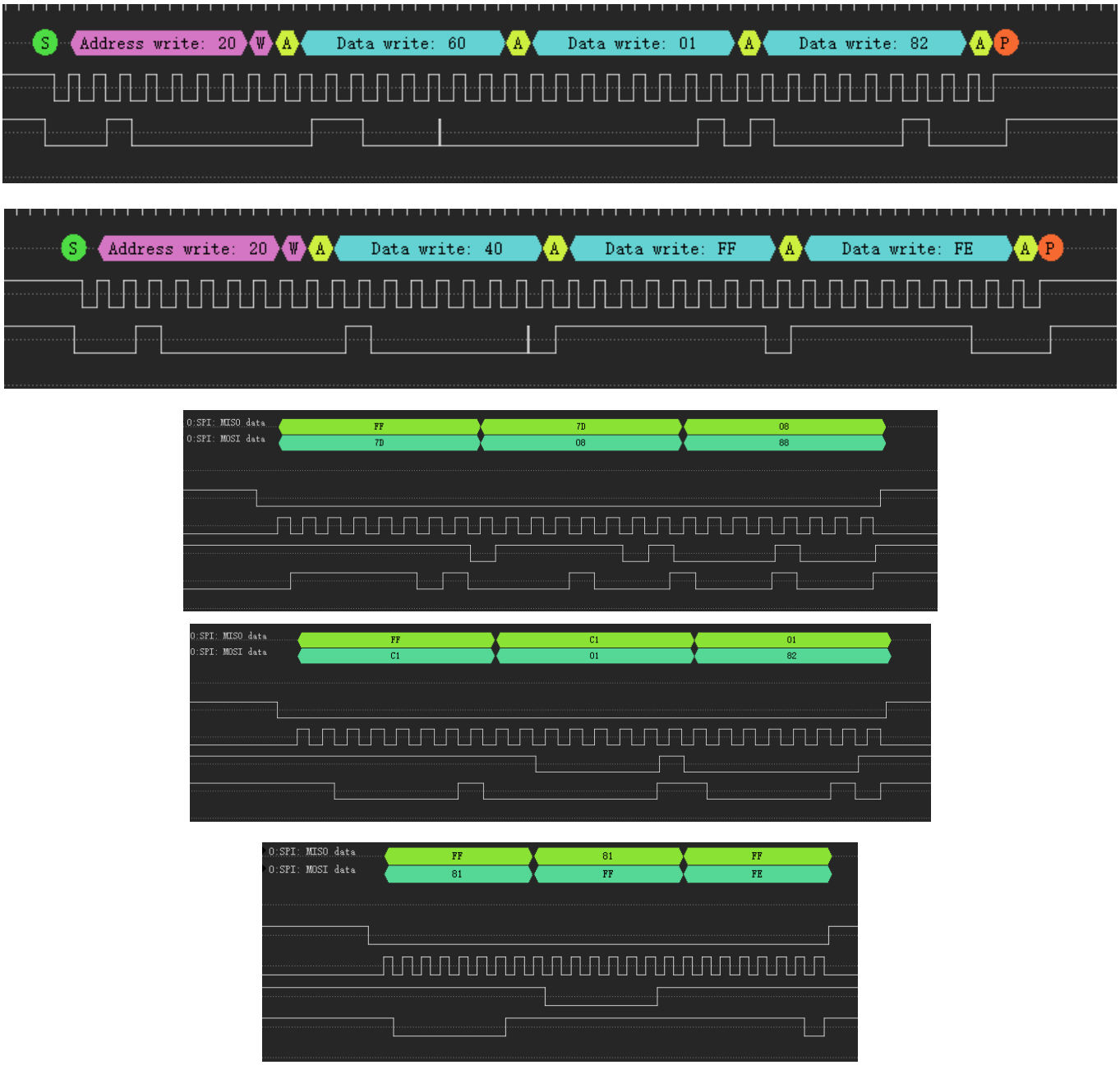
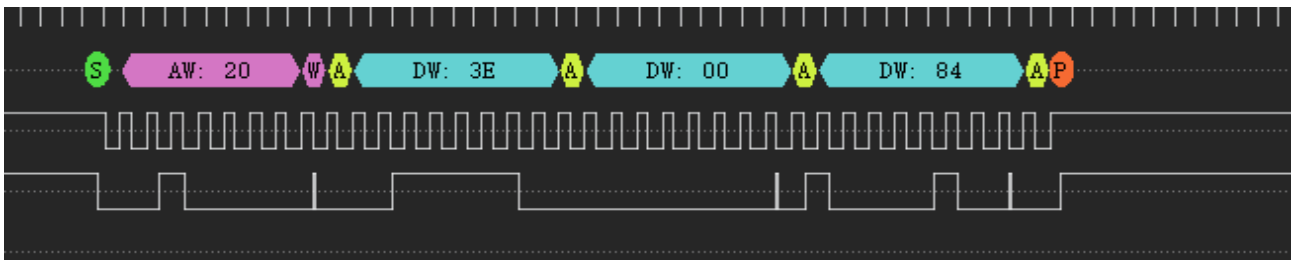
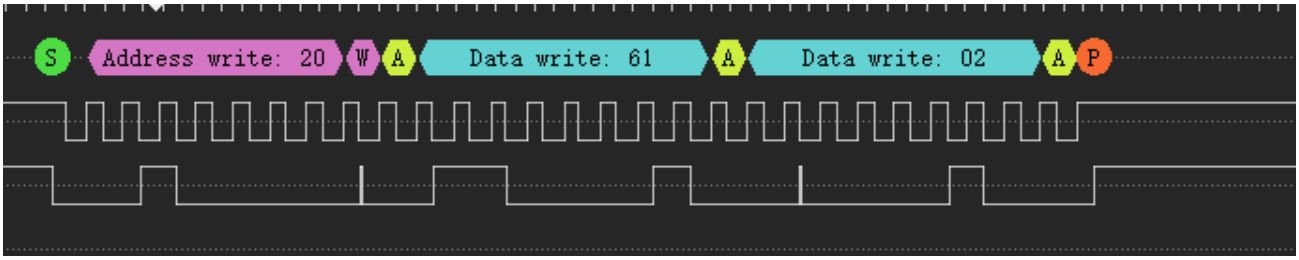


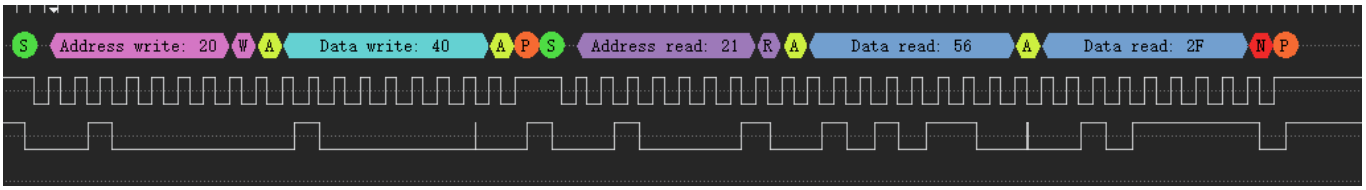
图5-2 写 RAM 寄存器示例

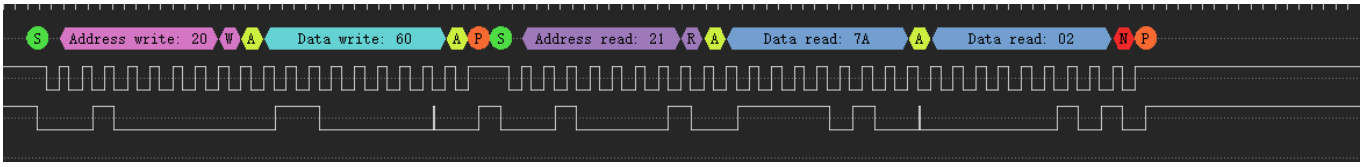
图 5-3 为读 cell 1 电压 gain 指令，将寄存器地址 0x8400（小端格式）写入 0x3E 入口地址中，0x61 寄存器中写入要读数据长度，然后从地址 0x40 的数据缓冲区中读取数据，从地址 0x60 的数据缓冲区读取校验和长度。

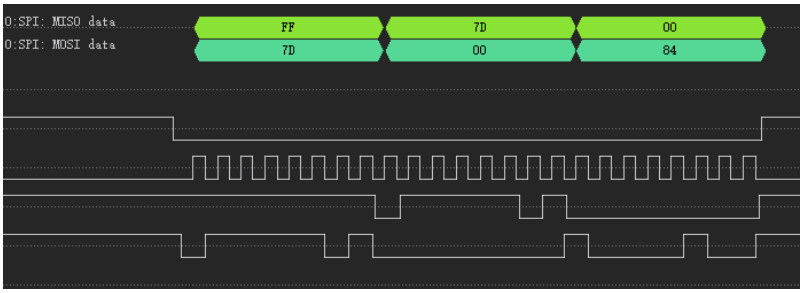


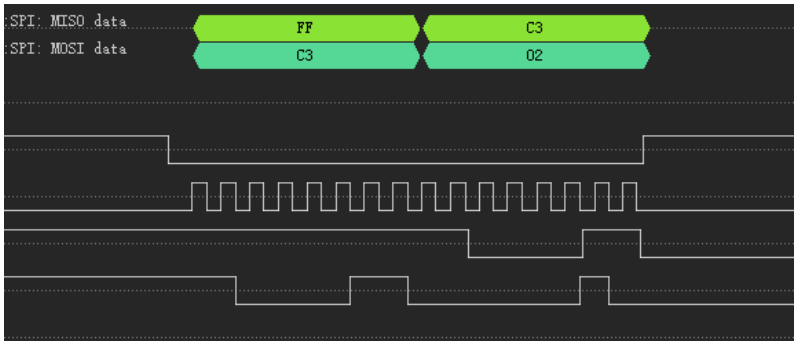
Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

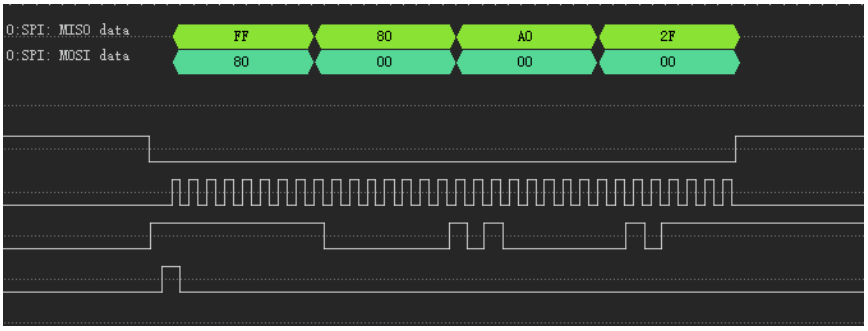












Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

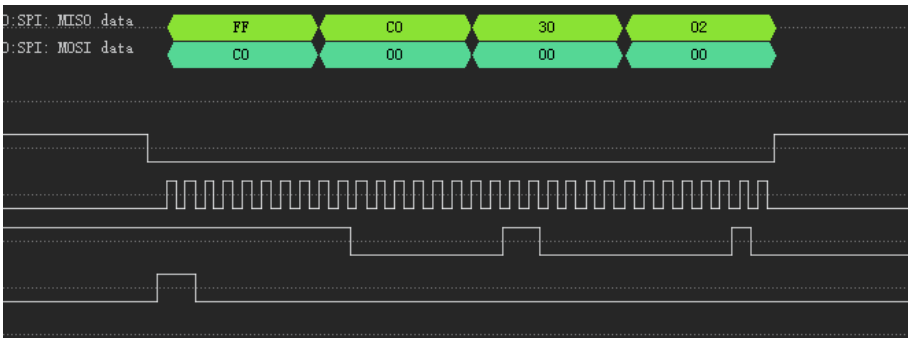
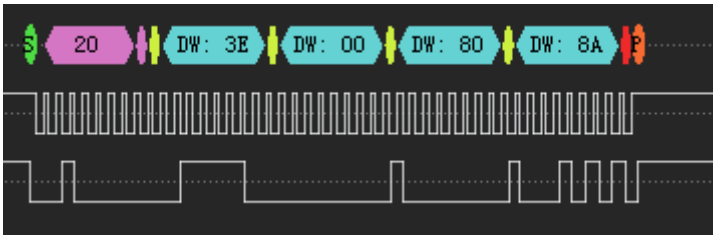
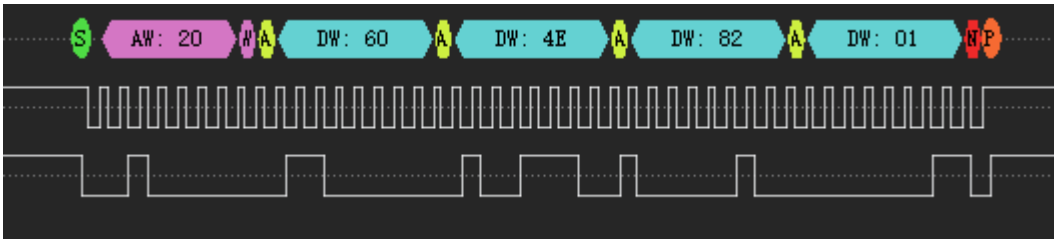


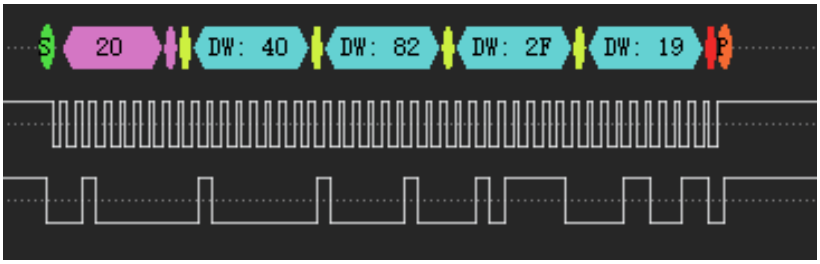
图5-3 读 RAM 寄存器示例

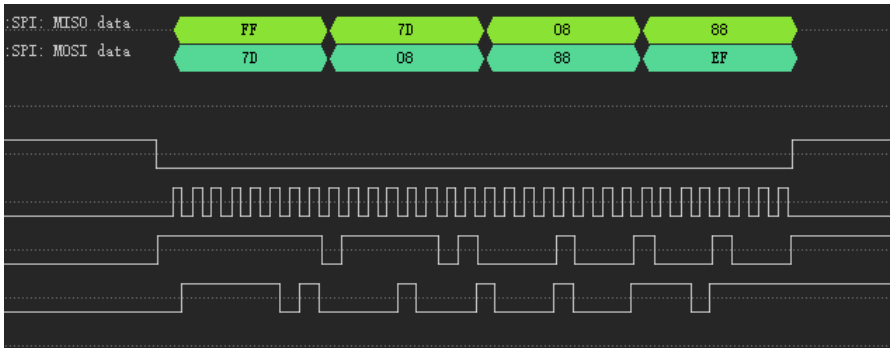
5.5 带 CRC 读写寄存器示例

图 5-4 为带 crc 写寄存器指令，每个通信指令最后一字节为 crc byte，第一条指令中的 crc = 0x8A，其是将 0x20、0x3E、0x00、0x80 代入到 CRC-8 中计算获得。I2C 第二条指令中的 0x4E 为写数据的 checksum 值，将写数据 0x20、0x60、0x4E、0x82 代入到 CRC 算法中计算。SPI 第一条指令中的 0xEF 为写数据 CRC 值，将写数据 0x7D、0x08、0x88 代入到 CRC 算法中计算。









Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

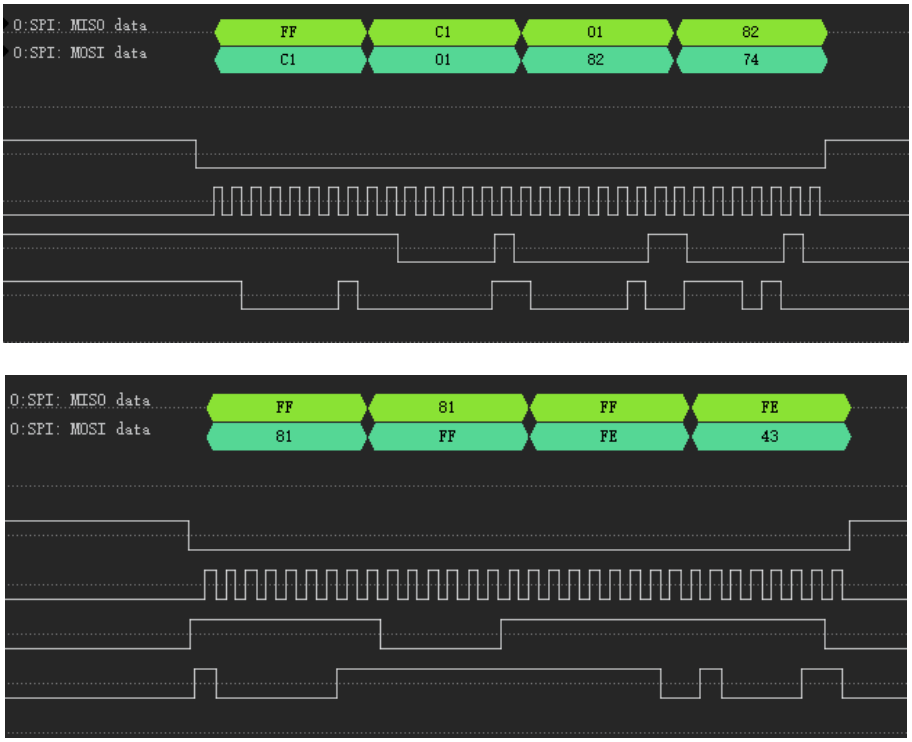
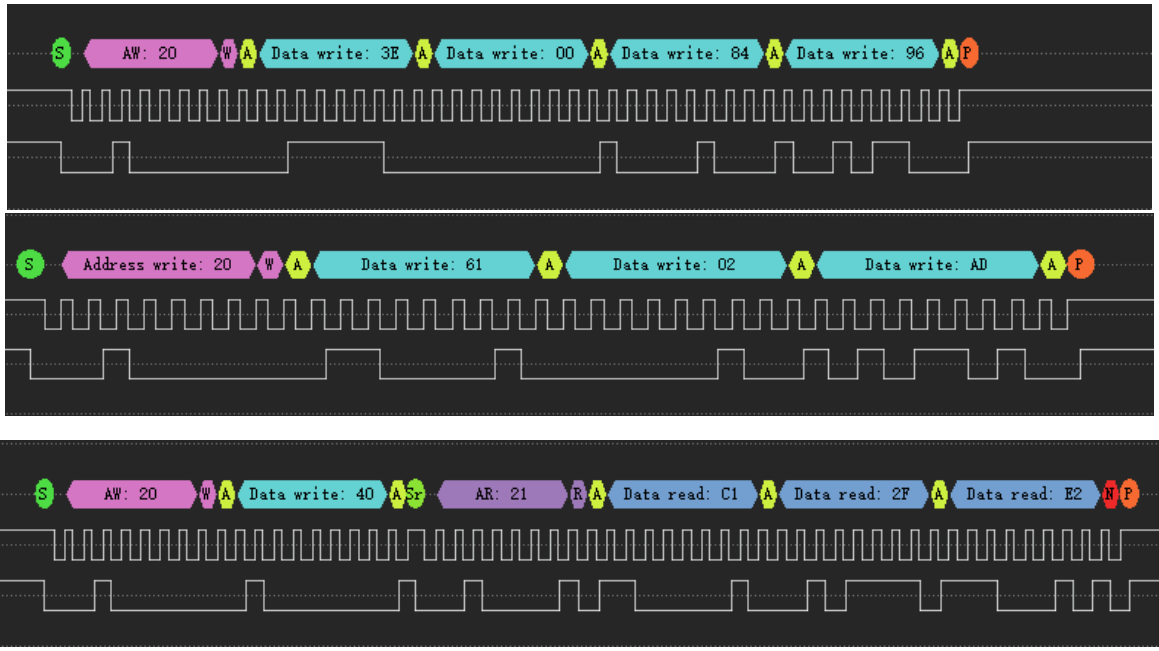


图5-4 带 CRC 写 RAM 寄存器示例

图 5-5 为带 crc 读寄存器指令，每个通信写指令最后一字节为 crc byte，其中第三条指令返回的寄存器数据，第四条指令返回寄存器数据的 check sum。第三条指令中的 crc = 0xE2，其是将 0x20、0x40、0x21、0xC1、0x2F 代入到 CRC-8 中计算获得。第四条指令中的 0x0F 为 AFE 返回数据的 checksum 值，将写数据 0xC1、0x2F 代入到 checksum 算法中计算，0x02 代表读数据的长度，0xD4 是 CRC 校验值。SPI 的第三条指令 0x46 为 CRC 的值，0x80、0x93 送入 CRC 计算的数据，0x2F 不计算 CRC。SPI 的第 4 条指令中 0x3D 为 0x93、0x2F 的 check sum 校验值，0x02 为读取长度，0x5E 为 0xC0 和 0x3D 的 CRC 校验和。



Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

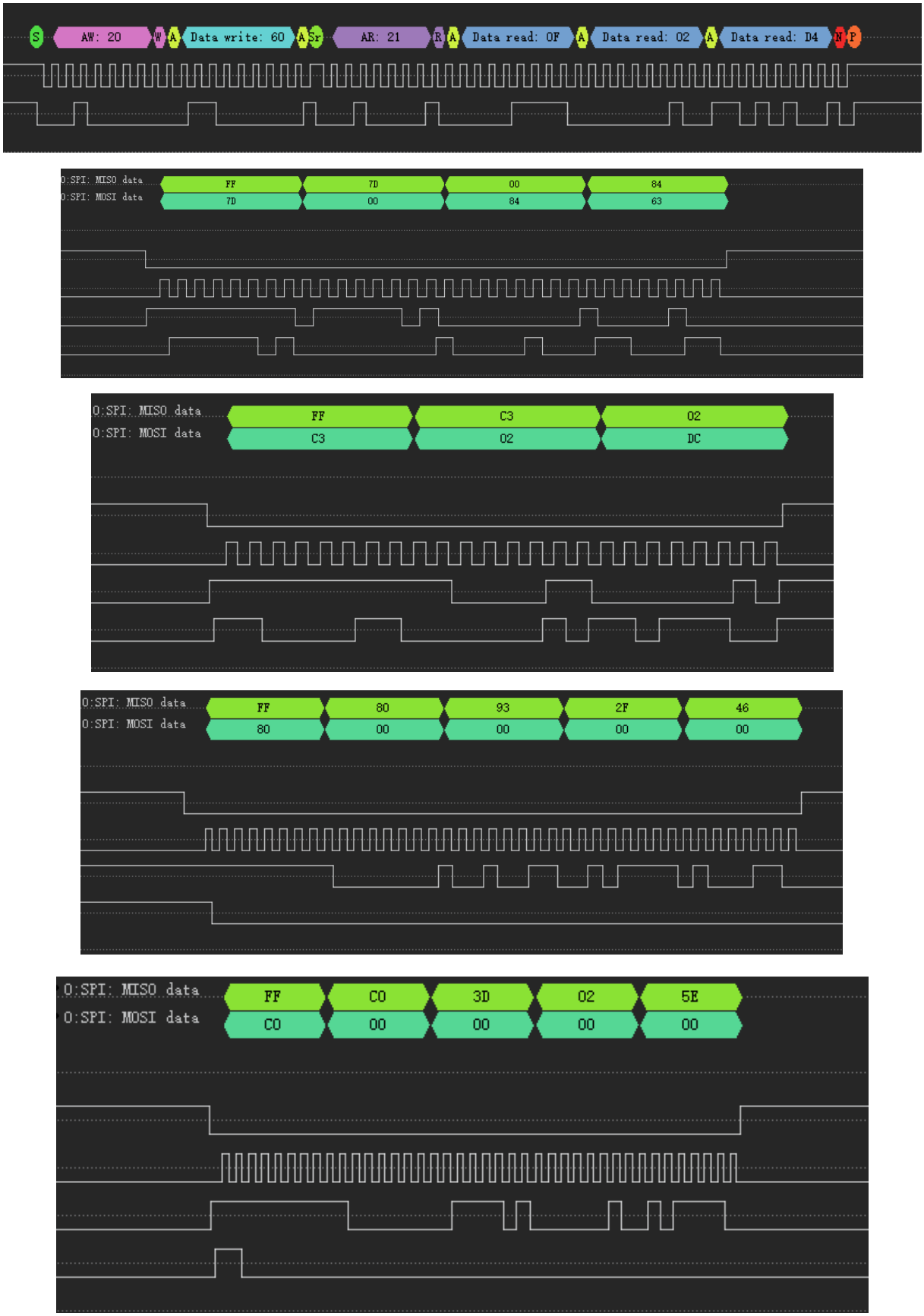


图5-5 带 CRC 读 RAM 寄存器示例

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

6 OTP 读写

OTP 寄存器为编程寄存器，存储器件出厂调试参数，同时包含一些客户定制开放的参数。OTP 读写指令与带数据子命令读写指令格式相同，只是寄存器地址为 0x9000-0x9FFF。器件的 OTP 只支持两次烧写，每次烧写内容会与原始值做异或处理，写入异或之后的值。OTP 写入只能从偶数地址写入，如果烧录奇数地址，奇数放在高位，偶数放在低位，使用偶数地址烧录。

寄存器与 OTP 映射关系如下：

1. Diag-CTL 寄存器地址 0X8E00 - 0X8E37, OTP 地址是 0X9000 - 0X9037；
2. SYS-CTL 寄存器地址 0X8000 - 0X804F, OTP 地址是 0X9038 - 0X9087；
3. PAD-CTL 寄存器地址 0X8200 - 0X8213, OTP 地址是 0X9088 - 0X909B；
4. ADC-CTL 寄存器地址 0X8400 - 0X846B, OTP 地址是 0X909C - 0X9107；
5. PROTECT-CTL 寄存器地址 0X8800 - 0X88BF, OTP 地址是 0X9108 - 0X91C7；
6. CB-CTL 寄存器地址 0X8C00 - 0X8C17, OTP 地址是 0X91C8 - 0X91DF

OTP 烧录前同样需要 seal，configure update mode，并且 OTP_WR_EN(寄存器 0x804A 写 0x2040)，写完 OTP 之后，需要将 OTP_WR_EN BIT 清除。在 OTP 烧写过程中，OTP 引脚会有电压输出，需要 OTP 引脚外接 2.2uF 的电容。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

7 ADC 较准

GD30BM2016 器件包含用于电芯电压 ADC 测量的出厂修正，这样即使客户未执行进一步校准，也可以优化电压测量性能。另外客户可以在生产线上进行校准，以进一步优化系统性能。ADC 校准包括电流电压通道的 gain 和 offset、温度系数等，GD30BM2016 大部分 gain 和 offset 等已经在工厂实现了校准，不需要用户另外再校准和配置。但电流采样电阻、温度 NTC 等是客户自行选择的，需要配置相应的寄存器。

7.1 电流校准

GD30BM2016 器件使用电流 CC Gain 配置值将 ADC 码值转换为电流，CC Gain 用户需要根据系统使用的检测电阻值进行调整，CC Gain 出厂有校准值，对应的电流采样电阻为 10mΩ。CC Gain 出厂校准值存在寄存器 Register: CC_GAIN –0x842c 中，CC_Gain reg value 默认值为 3000，根据使用的采样电阻阻值计算新的 CC Gain：CC Gain= 30 / 采样电阻/userA，采样电阻单位为 Ω，userA 单位根据用户使用的采样电阻和最大电流决定。

电流的 CC2 offset 出厂有校准值，若用户对 offset 有更严格的要求，可以自行回读电流值后校准电流的 OFFSET 值，通过写入 Reg: CC2_OFFSET -0x8432 寄存器修调 offset。 offset 值与 Reg: CC2_OFFSET 对应关系为 20 倍，例如发现负向偏移了 50mA，此时 Reg: CC2_OFFSET 为-1000，Reg:CC2_OFFSET 最高位为符号位，若为负值需采用补码计算，例如-1000 补码为 0xFC18。获取电流的 CC Gain 和 CC2 offset 后，按照下图写入对应的寄存器。

```
#----------------------------------电流校准-----------------------------------------------------
BM2016.Memory_write(0x842C ,0x1D4C, 2)        #配置cc_gain，受不同采样电阻、userA单位影响
```

7.2 温度校准

温度测量需要给出三个部分：设置 TS1，TS2, TS3 和 TS4 作为 NTC 温度输入，作为电池温度保护配置 TS0_1 CONFIG 和 TS2_3 为 0x55，设置上拉 18kΩ 电阻。18kΩ 上拉配合 Semitec 103AT 等热敏电阻一起使用，其在室温下的电阻为 10kΩ，温度结果转换与 BQ76952 处理一致，温度多项式系数在出厂前已校准好，数值如下所示。

18K TemperatureModel	COEFF A1 18K	60519
	COEFF A2 18K	16994
	COEFF A3 18K	63191
	COEFF A4 18K	1220

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

8 器件使用流程

8.1 器件使用注意事项

用户在使用 GD30BM2016 时，连接和配置必须按照下述说明使用，如果不按要求的作业，有可能会损坏保护板的元器件，从而导致器件不能保护电芯，造成严重的后果。

8.1.1 连接注意事项

连接步骤

1. 先连接电池组的动力线负极 BAT-到 PCB 板上的 BAT-；
2. 接着连接电池组的采样排线到 PCB 板上的电池采样口；
3. 最后连接电池组的动力线正极 BAT+到 PCB 板上的 BAT+；
4. 所有连接线安装好，再激活电池保护板开机，确认电池总电压与保护板输出电压相等。

断开保护板的步骤

1. 先将 PACK+至 PACK-的负载移除，然后通过指令关闭充放电 MOSFET；
2. 接着断开电池组正极的 BAT+连接线；
3. 然后拔下电池组的采样排线；
4. 最后断开电池组负极的 BAT-连接线。

8.1.2 配置注意事项

用户在使用 GD30BM2016 时，有以下配置事项需注意：

1. 用户需要设置 protect 寄存器中的 0x8808--VCELL_MODE[8]为 0，其他 bit 根据实际电池串的连接设置对应的使能;
2. Sys 寄存器中的 0x8008--COMM_IDLE_TIME[7:0]必须设置为非 0 值；
3. 若采用均衡功能，ADC 采样禁止配置为全速模式，0x8000--POWER_CONFIG[3:2]配置为非 0 值；
4. 带数据的子命令、RAM 寄存器读写命令与 BQ76952 不同，命令格式详情查看 *带数据子命令*和 *读写 RAM*。
5. GD30BM2016 与 BQ76952 的更多区别查看 GD30BM2016 Q&A Rev.A 文档。

8.2 初始化流程

器件的初始化主要是对于一些重要参数进行配置，其中包含系统参数（MOSFET 控制，采样速率，低功耗模式等），校准（如采样电阻值），引脚重新映射（NTC，GPIO 等），一次和二次保护参数以及均衡等。器件初始化流程主要包括 6 部分，具体如下：

1. 将器件进入 seal 模式，首先 unseal 后再进入 full access，进入 set_cfgupdate，此时才可以对于寄存器进行操作。
2. 系统寄存器配置，寄存器基地址 0x80xx，查看寄存器表 sysc 的 sheet，主要用来设置系统低功耗模式，配置 deepsleep，shutdown，sleep 下关于电荷泵，MOSFET 控制，时钟等参数。设备内部 LDO，REG0，REG1，REG2 的使能与电压, MOSFET 控制方式。
3. 保护相关参数，寄存器基地址 0x88xx，查看寄存器表格 protect 的 sheet。主要包含 cell mode（串数），COV，CUV，OCC，OCD1，OCD2，SCD，OTC，OTD，UTC，UTD 等相关一次保护，以及对应的二次保护参数，部分 MOSFET 保护的管关联控制。
4. 引脚映射，寄存器基地址 0x82xx，查看寄存器表格 pad 的 sheet 主要涉及到 TSx 的（上拉电阻和使能），CFETOFF，

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

DFETOFF，DDSG，DCHG，SPI_MOSI，ALERT 等引脚的 GPIO 或者 NTC 的配置状态。

5. 均衡相关的参数，寄存器基地址 0x8Cxx，查看寄存器表格 cb 的 sheet 主要包含均衡使能，均衡个数，均衡最小电压，均衡最小差值等。
6. 校准参数，寄存器基地址 0x84xx，查看寄存器表格 adc 的 sheet。

需要注意的是 GD30BM2016 大部分寄存器都已经在工厂实现了校准，不需要用户做校准。采样电阻是客户自行根据电流选择的，需要设置 cc2gain，上拉电阻温度系数需要和 NTC 类型和组织进行相应的配置，这个部分会推荐默认值，客户初始化自行加载。以上参数地址没有顺序要求，配置完这些参数后，直接执行 exit_cfgupdate，配置就可以生效。

8.3 初始化配置示例

初始化示例代码使用 python 编写，展示了 I2C 读写寄存器，其中写函数定义为 BM2016.Memory_write (command/reg_add, tx_data,len)，用户可参考下面示例来配置 GD30BM2016。

8.3.1 CONFIG_UPDATE 模式

GD30BM2016 默认为 SEALED 模式，该模式下主机可以读取大部分告警和数据，无法直接读取或更改数据寄存器的设置。如果想读取或者更改所有寄存器的设置，需要进入 UNSEALED 模式后再进入 FULLACCESS 模式，CONFIG_UPDATE 模式仅在 FULLACCESS 模式下可用。为了更改设置寄存器，用户需要在初始化开始进入 CONFIG_UPDATE 模式。

```
# ----------------- config_update_mode-----------------------------------
BM2016.write(0x3E, 0x032C, width=2)          #进入 seal mode
BM2016.Memory_write(0x0280, 0x5A5A5A5A, 4)
BM2016.Memory_write(0x0280, 0xA5A5A5A5, 4)
BM2016.write(0x3E, 0x0338, width=2)          #进入config update mode
```

8.3.2 保护寄存器配置

保护寄存器包含了电池串数、COV，CUV，OCC，OCD1，OCD2，SCD，OTC，OTD，UTC，UTD 等相关一次保护，以及对应的二次保护参数，部分 MOSFET 保护的管关联控制。下图是保护寄存器的示例配置，供用户参考。

```
# ----------------- 以下是进行 保护项目的设置-----------------------------------------------
BM2016.Memory_write(0x8808 ,0xFEFF, 2)       #Cell mode,  0xFEFF对应 16串模式，02EE对应10串模式
BM2016.Memory_write(0x880C ,0xFC, 1)         #Enable Protection A, Enable SCD, OCD1, OCD2, OCC, CUV, COV
BM2016.Memory_write(0x880D ,0x33, 1)         #Enable Protection B, disable FET OT, disable chip ot/ut, enable battery OT/UT for
BM2016.Memory_write(0x880E ,0x00, 1)         #Enable Protection C, SCDL disable, other disable

BM2016.Memory_write(0x881C ,0x3E8, 2)        #BODY DIODE TH 1A，单位1mA

BM2016.Memory_write(0x8824 ,0x09, 1)         #FET初始打开，PDSG 不使能，HOST 不控制 FET，SLEEP 不可以打开CHG，MOSFET串口模式
BM2016.Memory_write(0x8825 ,0x0D, 1)         # CP ON when sleep, normal and CP 电压11V

BM2016.Memory_write(0x882C ,0x007C, 2)       #CUV阈值 设置为 2.5V
BM2016.Memory_write(0x882E ,0x0033, 2)       #CUV阈值延迟 设置为200ms
BM2016.Memory_write( 0x8830 ,0x0019, 2)      #CUV 恢复阈值 3V，0.5V的迟滞 20mv step

BM2016.Memory_write(0x8832 ,0x00D4, 2)       #COV阈值调整，过压阈值设置为 4.26V
BM2016.Memory_write(0x8834 ,0x0033, 2)       #COV阈值延迟 设置为 200ms
BM2016.Memory_write(0x8836 ,0x0050, 2)       #COV 恢复阈值 4.1V，0.16V的迟滞 20mv Step

BM2016.Memory_write(0x883A ,0x0005, 2)       #OCC 配置，3A过流充电保护，采样电阻4mohm，  (N+1)*2mv/采样电阻值
BM2016.Memory_write(0x883C ,0xFD, 1)         #OCC 延迟保护设置为 1S
BM2016.Memory_write(0x8840 ,0x0708, 2)       #OCC 保护恢复阈值，单位mA，1.8A

BM2016.Memory_write( 0x8844 ,0x0003, 2)      #OCD1 配置，2A过放电保护，采样电阻4mohm
BM2016.Memory_write(0x8846 ,0xFD, 1)         #OCD1 延迟保护设置为 1S
BM2016.Memory_write( 0x8848 ,0x0007, 2)      #OCD2 配置，4A过放电保护，采样电阻4mohm
BM2016.Memory_write(0x884A ,0x7E, 1)         #OCD2 延迟保护设置为 0.5S
BM2016.Memory_write(0x8852 ,0xFF37, 2)       #OCD 恢复阈值，设置为-200mA，过载消失后，超过恢复时间就恢复

BM2016.Memory_write( 0x884B ,0x02, 1)        #SCD 配置，40mv 过放电保护，采样电阻4mohm，  电流值为10A
BM2016.Memory_write(0x884C ,0x02, 1)         #SCD 延迟保护设置为 31.25us
BM2016.Memory_write(0x884D ,0xFF, 2)         #SCD 短路恢复 256S 如果负载移除后
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

```
BM2016.Memory_write( 0x885A ,0x02, 1)        #SCDl 配置, 2次
BM2016.Memory_write(0x885B ,0x0A, 1)         #SCDl 延迟减少 10S
BM2016.Memory_write(0x885C ,0x0A, 1)         #SCDl 延迟恢复 10S
BM2016.Memory_write(0x885E ,0x2710, 2)       #SCDl 短路恢复 10A


BM2016.Memory_write( 0x8860 ,0x3C, 1)        #OTC 配置, 充电过温阈值 60℃
BM2016.Memory_write(0x8861 ,0x05, 1)         #OTC delay 配置时间为5S
BM2016.Memory_write( 0x8862 ,0x2D, 1)        #OTC 恢复配置阈值 45℃
BM2016.Memory_write(0x8863,0x41, 1 )         #OTD 配置, 充电过温阈值 65℃
BM2016.Memory_write( 0x8864 ,0x05, 1)        #OTD delay 配置时间为5S
BM2016.Memory_write(0x8865 ,0x32, 1)         #OTD 配置, 充电过温阈值 50℃

BM2016.Memory_write( 0x886C ,0xFB, 1)         #UTC 配置, 充电低温阈值 -5℃
BM2016.Memory_write(0x886D ,0x02, 1)          #UTC delay 配置时间为2S
BM2016.Memory_write( 0x886E ,0x01, 1)         #UTC 恢复配置阈值 1℃
BM2016.Memory_write(0x886F,0xD3, 1)           #UTD 配置, 放电过温阈值 -45℃
BM2016.Memory_write(0x8870,0x02, 1)           #UTD delay 配置时间为2S
BM2016.Memory_write(0x8871,0xD8, 1)           #UTC 恢复配置阈值 -40℃


BM2016.Memory_write(0x8874,0xF1, 1)
BM2016.Memory_write(0x8875,0x0A, 1)           #RECOVERY DELAY
```

8.3.3 引脚复用功能

GD30BM2016 器件设定了多个多功能引脚。多功能管脚包括 TS1,TS2, TS3, CFETOFF, DFETOFF, ALERT, SPI_MOSI,SPI_CS,DCHG, DDSG，每个引脚可以独立配置成不同的工作模式。下面是 DDSG 和 TSX 复用案例，用户根据实际复用情况进行配置。Pin 复用寄存器基地址 0x8200，具体地址请查看寄存器表 pad 的 sheet。

```
#----------------------------------TS 相关的设置-----------------------------------------------------
BM2016.Memory_write(0x8208 ,0x02, 1)          #DDSG 作为输出只是 DSG引脚状态， OTP需要从偶数地址写
BM2016.Memory_write(0x8209 ,0x50, 1)          #TS1作为Tsensor
BM2016.Memory_write(0x820a ,0x00, 1)          #TS2 and TS3 未使用
BM2016.Memory_write(0x8210, 0x80, 1)          #PAD CFG配置, 48pin mode， FUSE引脚作为输出， TS 18K上拉，没有BUSBAR,
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

8.3.4 ADC 配置

ADC 寄存器主要是配置电流电压通道的 gain 和 offset、温度系数等，GD30BM2016 大部分增益和 offset 等已经在工厂实现了校准，不需要用户另外再校准和配置。但电流采样电阻、温度 NTC 等是客户自行选择的，需要配置相应的寄存器，下图给出了 ADC 配置的简单示例。

```
#-----------------------------------TS温度系数配置-----------------------------------
BM2016.Memory_write(0x8448 ,60519 , 2)          # 配置内部上拉电阻系数
BM2016.Memory_write(0x844A,16994 ,2)
BM2016.Memory_write(0x844C,63191 ,2)
BM2016.Memory_write(0x844E,1220 ,2)
```

8.3.5 系统寄存器配置

系统寄存器包含了系统模式、LDO 设置、MOSFET 控制、时钟等参数，用户需根据自身需求，配置相应的寄存器。寄存器基地址 0x8000，具体地址请查看寄存器表 sysc 的 sheet。

```
#-----------------------------------TS温度系数配置-----------------------------------------------
BM2016.Memory_write(0x8448 ,60519 , 2)          # 配置内部上拉电阻系数
BM2016.Memory_write(0x844A,16994 ,2)
BM2016.Memory_write(0x844C,63191 ,2)
BM2016.Memory_write(0x844E,1220 ,2)

# ----------------- 以下是系统的设置-----------------------------------------------
BM2016.Memory_write(0x8000 ,0x0B86, 2)          #ADC降速 1/8, DEEPSLEEP模式下，关闭reg1和reg2
BM2016.Memory_write(0x8002 ,0x00, 2)            #默认关闭REG1和REG2，  如果不烧写，系统 REG1使能5V
BM2016.Memory_write(0x8003 ,0x00, 2)            #关闭PRE-REGULATOR，默认打开
BM2016.Memory_write(0x8008 ,0x05, 2)            #COMM_IDLE_TIME不为0
BM2016.Memory_write(0x800D ,0x00, 2)            #默认关闭REG2，  使能REG1使能 3.3V，


BM2016.Memory_write(0x8012, 0x00, 1)            #AUTO SHUTDOWN，  OTP需要从偶数地址写
BM2016.Memory_write(0x8013, 0x50, 1)            #打开充放电MOS FET

BM2016.Memory_write(0x8017, 0xff, 1)            #sleep模式下最小时间间隔
```

8.3.6 均衡寄存器配置

GD30BM2016 支持电芯之间的均衡，用户可自行配置均衡开启和关闭的条件实现自动均衡，另外用户也可以通过主机发送命令实现手动均衡。下图为电芯均衡示例，主要包括均衡电压、电流、温度等。

```
# ----------------- 以下是进行 均衡的的设置-----------------------------------------------
BM2016.Memory_write(0x8C00 ,0xC9 , 1)    #CB ENABLE
BM2016.Memory_write(0x8C01 ,0xEC , 1)    #min cell temp -20℃
BM2016.Memory_write(0x8C02 ,0x3C , 1)    #max cell temp 60℃
BM2016.Memory_write(0x8C03 ,0x46 , 1)    #max int temp 70℃
BM2016.Memory_write(0x8C04 ,0x14 , 1)    #max charge timer 20s
BM2016.Memory_write(0x8C05 ,0x0F , 1)    #max balance cell

BM2016.Memory_write(0x8C06 ,0x0BB8 , 2)  # 充电均衡 最小电压 3000mv
BM2016.Memory_write(0x8C08 ,0x28 , 1)  # 充电均衡开始条件 最小cell和最大cell电压 40mv
BM2016.Memory_write(0x8C09 ,0x14 , 1)  # 充电均衡结束条件  最小cell和最大cell电压 20mv

BM2016.Memory_write(0x8C0A ,0x0BB8 , 2)  # 静置均衡 最小电压 3000mv
BM2016.Memory_write(0x8C0C ,0x28 , 1)  # 静置均衡开始条件 最小cell和最大cell电压 40mv
BM2016.Memory_write(0x8C0D ,0x14 , 1)  # 静置均衡结束条件 最小cell和最大cell电压 20mv
```


Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

8.3.7 初始化配置生效

以上寄存器的均在 CONFIG_UPDATE 模式下更改，用户在更改完成后，主机必须发送 EXIT_CFG_UPDATE 命令退出该模式，退出后器件根据最新的配置正常运行，执行所有测量和保护监控，EXIT_CFG_UPDATE 命令如下图所示。

```
BM2016.write(0x3E, 0x033C, width=2)     #退出config_update
```

8.4 采样数据和寄存器状态读取

执行完初始化流程后，GD30BM2016 进入正常工作模式，对电池包电压、电流、状态进行监控，同时检测到电池异常执行保护动作。用户通过发送 dircmd 命令，读取常规的 cell 电压、电流、温度、告警等信息。具体的直接命令查阅*直接命令表*章节。Indir cmd with data 这部分主要是一些电压，电流和温度等信息的中间 ADC 原始码值，用户使用这些码值自行计算一些校准参数，用于工厂产线校准。

另外用户也可发送子命令控制器件，主要是是对于一些操作的一键直达。例如用户可以发送模式切换、MOSFET 直接控制、复位、故障模式退出等常用功能，具体命令查阅*不带数据子命令表*和***错误!未找到引用源。***章节。

```
# ----------------- 读cell电压-----------------------------------------------
    CELL1 = BM2016.read(0x14, 2)
    CELL2 = BM2016.read(0x16, 2)
    CELL3 = BM2016.read(0x18, 2)
    CELL4 = BM2016.read(0x1A, 2)
    CELL5 = BM2016.read(0x1C, 2)
    CELL6 = BM2016.read(0x1E, 2)
    CELL7 = BM2016.read(0x20, 2)
    CELL8 = BM2016.read(0x22, 2)
    CELL9 = BM2016.read(0x24, 2)
    CELL11 = BM2016.read(0x28, 2)
    CELL12 = BM2016.read(0x2A, 2)
    CELL13 = BM2016.read(0x2C, 2)
    CELL14 = BM2016.read(0x2E, 2)
    CELL15 = BM2016.read(0x30, 2)
    CELL16 = BM2016.read(0x32, 2)

# ----------------- 读CC2 电压-----------------------------------------------
    CC2= BM2016.read(0x3A, 2)

# ----------------- 读TS引脚温度-----------------------------------------------
    TS1 = BM2016.read(0x70, 2)
    TS2 = BM2016.read(0x72, 2)
    TS3 = BM2016.read(0x74, 2)

# ----------------- 获取告警和故障等状态-----------------------------------
    Alert_Value = BM2016.read(0x02, 1)
    Status_Value = BM2016.read(0x03, 1)
    Alarm_Value = BM2016.read(0x62, 1)
    FETStatus_Value = BM2016.read(0x7F, 1)
```

8.5 采样数据转换计算

电芯电压、电池包电压等测量值计算、电流、温度码值转换公式在本章节中列出，其中 CC2 回读结果为 user A，如果要更改 userA 需要同步修改 CC GAIN。比如如果修改 userA 为 8mA，此时需要将原来 CC GAIN 配置成原来 1/8，此时读数才正确。另外以下计算，用户可以不用关心，使用直接命令读取出来的就是最后的计算的结果。

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

- CELL_VOLTAGE 计算：

$$V_{step} = \frac{5 * V_{REF1}}{2^{23}} \approx \frac{5 * 1.212V}{2^{23}} \approx 0.722uV$$

$$V_{cell} = Count_{cell\ voltage} * V_{step} \approx Count_{cell\ voltage} * 0.722uV$$

- I_VOLTAGE 计算：

$$I_{step} = \frac{V_{REF2}}{5 * 2^{23}} \approx \frac{1.212V}{5 * 2^{23}} \approx 28.896nV$$

$$V_{current} = Count_{current} * I_{step} \approx Count_{current} * 28.896nV$$

- CC2 电流最高位为符号位，如果发现 CC2 Current 返回值大于 32767，则其为负电流需要转为补码后再乘以 userA.

$$Current_{cc2} = Count_{cc2} * userA$$

STACK、PACK、LD VOLTAGE 回读结果为 userV，系统默认 userV 为 8mV，用户也可以通过 protect 寄存器中的 0x8807 DA_CONFIGURATION [2]设置 userV 的单位。

- STACK_VOLTAGE 计算：

$$V_{stack} = Count_{stack} * \text{user}V$$

- PACK_VOLTAGE 计算：

$$V_{pack} = Count_{pack} * \text{user}V$$

- LD_VOLTAGE 计算：

$$V_{LD} = Count_{LD} * \text{user}V$$

温度读取值使用 DIRECT_COMMAND，所有温度的回读单位是 0.1°C，如果发现温度返回值返回值大于 32767，温度低于 0 摄氏度需要先转化为补码再计算。

$$temp = Count_{temp} * 0.1℃$$

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

9 历史版本

版本号	描述	日期
A	初始发布	2024 年 3 月 20
B	更新 CRC 和 check sum 的详细描述	2024 年 10 月 30
C	增加 SPI 通信描述，更新器件使用流程	2025 年 3 月 20
D	更新电流计算，删除子命令表	2026 年 3 月 20

Copyright © 2025, GigaDevice Semiconductor Inc.
All Rights Reserved.

Important Notice

This document is the property of GigaDevice Semiconductor Inc. and its subsidiaries (the "Company"). This document, including any product of the Company described in this document (the “Product”), is owned by the Company according to the laws of the People’s Republic of China and other applicable laws. The Company reserves all rights under such laws and no Intellectual Property Rights are transferred (either wholly or partially) or licensed by the Company (either expressly or impliedly) herein. The names and brands of third party referred thereto (if any) are the property of their respective owner and referred to for identification purposes only.

The Company makes no representations or warranties of any kind, express or implied, with regard to the merchantability and the fitness for a particular purpose of the Product, nor does the Company assume any liability arising out of the application or use of any Product described in this document. Any information provided in this document is provided only for reference purposes. It is the sole responsibility of the user of this document to determine whether the Product is suitable and fit for its applications and products planned, and properly design, program, and test the functionality and safety of its applications and products planned using the Product. Unless otherwise expressly specified in the datasheet of the Product , the Product is designed, developed, and/or manufactured for ordinary business, industrial, personal, and/or household applications only, and the Product is not designed or intended for use in (i) safety critical applications such as weapons systems, nuclear facilities, atomic energy controller, combustion controller, aeronautic or aerospace applications, traffic signal instruments, pollution control or hazardous substance management; (ii) life-support systems, other medical equipment or systems (including life support equipment and surgical implants); (iii) automotive applications or environments, including but not limited to applications for active and passive safety of automobiles (regardless of front market or aftermarket), for example, EPS, braking, ADAS (camera/fusion), EMS, TCU, BMS, BSG, TPMS, Airbag, Suspension, DMS, ICMS, Domain, ESC, DCDC, e-clutch, advanced-lighting, etc.. Automobile herein means a vehicle propelled by a self-contained motor, engine or the like, such as, without limitation, cars, trucks, motorcycles, electric cars, and other transportation devices; and/or (iv) other uses where the failure of the device or the Product can reasonably be expected to result in personal injury, death, or severe property or environmental damage (collectively "Unintended Uses"). Customers shall take any and all actions to ensure the Product meets the applicable laws and regulations. The Company is not liable for, in whole or in part, and customers shall hereby release the Company as well as its suppliers and/or distributors from, any claim, damage, or other liability arising from or related to all Unintended Uses of the Product. Customers shall indemnify and hold the Company, and its officers, employees, subsidiaries, affiliates as well as its suppliers and/or distributors harmless from and against all claims, costs, damages, and other liabilities, including claims for personal injury or death, arising from or related to any Unintended Uses of the Product.

Information in this document is provided solely in connection with the Product. The Company reserves the right to make changes, corrections, modifications or improvements to this document and the Product described herein at any time without notice. The Company shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them. Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2025 GigaDevice – All rights reserved